

PROYEK BESAR ALGORITMA PEMROGRAMAN



Dosen Pengampu :

Lutfiah Maharani Siniwi, S.Stat., M.Stat.

disusun oleh :

KELOMPOK G - STATISTIKA A

1. Aurn Nathan Fauzia (K1D025016)
2. Rizky Aulia Putri (K1D025029)
3. Lanang Tegar Harsono (K1D025040)
4. Muhamad Aji Firmansyah (K1D025041)

PROGRAM STUDI STATISTIKA

MATA KULIAH ALGORITMA PEMROGRAMAN

UNIVERSITAS JENDERAL SOEDIRMAN

PURWOKERTO

2026

DAFTAR ISI

DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan <i>Project</i>	2
1.3 Ruang Lingkup <i>Project</i>	2
BAB II ANALISIS MASALAH.....	4
2.1 Soal <i>Project</i>	4
2.1.1 Program 1	4
2.1.2 Program 2	4
2.1.3 Program 3	4
2.1.4 Program 4	5
2.1.5 Program 5	5
2.1.6 Program 6	5
2.2 Tabel Input-Proses-Output.....	5
2.3 Batasan Masalah	8
2.4 Pembagian Tugas	10
BAB III PSEUDOCODE DAN FLOWCHART	11
3.1 Pseudocode	11
3.1.1 Program 1	11
3.1.2 Program 2	11
3.1.3 Program 3	12
3.1.4 Program 4	13
3.1.5 Program 5	14
3.1.6 Program 6	15
3.2 Flowchart	16
3.2.1 Program 1	16
3.2.2 Program 2	17
3.2.3 Program 3	18
3.2.4 Program 4	19
3.2.5 Program 5	19

3.2.6 Program 6	21
BAB IV IMPLEMENTASI PROGRAM.....	22
4.1 Program 1.....	22
4.1.1 Code R :.....	22
4.1.2 Code Python :	23
4.2 Program 2.....	25
4.2.1 Code R :.....	25
4.2.2 Code Python	25
4.3 Program 3.....	25
4.3.1 Code R.....	26
4.3.2 Code Python	28
4.4 Program 4.....	29
4.4.1 Code R.....	29
4.4.2 Code Python	31
4.5 Program 5.....	32
4.5.1 Code R.....	32
4.5.2 Code Python :	34
4.6 Program 6.....	36
4.6.1 Code R :.....	36
4.6.2 Code Python :	37
BAB V PENJELASAN LINE-BY-LINE CODE.....	40
5.1 Penjelasan Line-By-Line Code Program 1	40
5.1.1 Penjelasan Line-By-Line Code R.....	40
5.1.2 Penjelasan Line-By-Line Code Python	42
5.2 Penjelasan Line-By-Line Code Program 2	44
5.2.1 Penjelasan Line-By-Line Code R.....	44
5.2.2 Penjelasan Line-By-Line Code Python	45
5.3 Penjelasan Line-By-Line Code Program 3	46
5.3.1 Penjelasan Line-By-Line Code R.....	46
5.3.2 Penjelasan Line-By-Line Code Python	48
5.4 Penjelasan Line-By-Line Code Program 4	49
5.4.1 Penjelasan Line-By-Line Code R.....	49
5.4.2 Penjelasan Line-By-Line Code Python	52

5.5 Penjelasan Line-By-Line Code Program 5	54
5.5.1 Penjelasan Line-By-Line Code R.....	54
5.5.2 Penjelasan Line-By-Line Code Python	58
5.6 Penjelasan Line-By-Line Code Program 6	62
5.6.1 Penjelasan Line-By-Line Code R.....	62
5.6.2 Penjelasan Line-By-Line Code Python	64
BAB VI PENGUJIAN PROGRAM	66
6.1 Pengujian Program 1.....	66
6.1.2. <i>Screenshot</i> Hasil <i>Running</i>	66
6.1.3. Interpretasi <i>Output</i>	67
6.2 Pengujian Program 2.....	68
6.2.1 Tabel Pengujian	68
6.2.2. <i>Screenshot</i> Hasil <i>Running</i>	69
6.2.3. Interpretasi <i>Output</i>	69
6.3 Pengujian Program 3.....	70
6.3.2. <i>Screenshot</i> Hasil <i>Running</i>	70
6.3.3. Interpretasi <i>Output</i>	72
6.4 Pengujian Program 4.....	72
6.4.1 Tabel Pengujian	72
6.4.2. <i>Screenshot</i> Hasil <i>Running</i>	73
6.4.3. Interpretasi <i>Output</i>	75
6.5 Pengujian Program 5.....	75
6.5.1. Tabel pengujian.....	76
6.5.2. <i>Screenshot</i> Hasil <i>Running</i>	77
6.5.3. Interpretasi <i>Output</i>	77
6.6 Pengujian Program 6.....	78
6.6.1. Tabel pengujian.....	78
6.6.2. <i>Screenshot</i> Hasil <i>Running</i>	79
6.6.3. Interpretasi <i>Output</i>	80
BAB VII PENUTUP	81
7.1 Kesimpulan	81
7.2 Kendala	81
7.3 Saran	82

LAMPIRAN	83
1. Link Github:.....	83
2. Link Video Demo:	83
3. Link Penggunaan Ai:	83

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini menuntut penyelesaian masalah dilakukan secara cepat, akurat, dan otomatis melalui bantuan komputasi. Kemampuan merancang program algoritma dan menerjemahkannya ke dalam bahasa pemrograman telah menjadi keterampilan dasar yang sangat dibutuhkan untuk menangani pengolahan informasi dan data yang semakin kompleks. Di dalam praktiknya, penyelesaian persoalan komputasional dan pemodelan matematis membutuhkan alur berpikir yang sangat terstruktur. Proses ini memerlukan penerapan konsep algoritma yang logis, mulai dari identifikasi input, proses perhitungan, hingga penentuan *output*, sebelum akhirnya dieksekusi oleh mesin. Untuk menggambarkan proses perencanaan alur algoritma, dibuat flowchart untuk memvisualisasikan logika program dan pseudocode yang menjadi jembatan antara bahasa manusia dan *sintaks* bahasa program. Tanpa perancangan algoritma yang baik, sebuah instruksi program tidak akan berjalan secara efisien dan kehilangan arah.

Setelah algoritma dirancang secara logis, algoritma tersebut diimplementasikan ke dalam bahasa pemrograman yang relevan. Bahasa R dan Python dipilih karena keduanya sangat andal untuk menyelesaikan berbagai kasus komputasi, mulai dari manipulasi *string* sederhana hingga perhitungan matematis yang lebih rumit. Walaupun kedua bahasa ini memiliki aturan *sintaks* dan gaya penulisan yang berbeda, dengan alur logika yang sama dan setara, penyusunan kodenya akan menghasilkan *output* yang sama. Pada proses pengembangan program dalam bahasa Python, implementasi dan pengujian memanfaatkan lingkungan komputasi berbasis Google Colaboratory, sedangkan pada bahasa R digunakan RStudio.

Tugas *project* akhir kelompok ini dirancang untuk mengaplikasikan pemahaman konsep dasar algoritma tersebut ke dalam persoalan komputasional yang nyata. Mahasiswa diwajibkan menyelesaikan enam studi kasus, meliputi analisis teks, format *string*, akar persamaan kuadrat, ekstraksi data NIP ASN, klasifikasi *cluster* 3 dimensi, hingga interval konfidensi proporsi. Melalui pengerjaan *project* ini, pemahaman teoretis mengenai alur logika akan ditransformasikan menjadi barisan *source code* yang fungsional, terstruktur, dan tervalidasi dengan baik. Oleh karena itu, penyusunan laporan ini diharapkan mampu mendokumentasikan seluruh proses analisis dan komputasi secara sistematis untuk membuktikan pemahaman yang utuh terhadap algoritma dan pemrograman.

1.2 Tujuan *Project*

Tugas *project* akhir kelompok ini disusun untuk mencapai beberapa tujuan, yaitu sebagai berikut:

1. Menganalisis permasalahan komputasional dengan mengidentifikasi input, proses, *output*, serta batasan masalah secara mendetail.
2. Menyusun rancangan algoritma penyelesaian masalah dalam bentuk narasi, pseudocode, dan flowchart yang runtut dan logis.
3. Mengimplementasikan algoritma ke dalam dua bahasa pemrograman, yaitu R dan Python, serta membandingkan struktur logika antara keduanya pada kasus komputasi yang sama.
4. Menerapkan berbagai konsep dasar pemrograman seperti variabel, tipe data, *string*, percabangan, fungsi, *tuple/list/vector*, dan operasi matematika.
5. Melatih kemampuan validasi input dan penanganan kondisi khusus (*error handling*) pada jalannya program.
6. Menguji program menggunakan beberapa skenario data secara sistematis dan menjelaskan setiap baris kode secara logis, bukan sekadar menerjemahkan *sintaks*.
7. Membangun kerja sama tim yang kolaboratif dengan mendistribusikan kontribusi secara adil menggunakan *repository* GitHub.

1.3 Ruang Lingkup *Project*

Ruang lingkup pada *project* ini mencakup keseluruhan proses penyelesaian masalah, mulai dari tahap analisis hingga pengujian akhir, yang difokuskan pada enam studi kasus komputasional. Batasan dan cakupan pengerjaannya meliputi:

1. Penyelesaian enam studi kasus.

Meliputi program analisis teks (perhitungan kata dan kalimat), pencetakan format *string*, pencarian akar persamaan kuadrat, pengolahan NIP ASN untuk dicari tanggal lahir, penentuan klasifikasi *cluster* 3 dimensi, dan perhitungan interval konfidensi proporsi.

2. Analisis dan perancangan

Meliputi penyusunan tabel Input-Proses-Output, identifikasi batasan masalah, penentuan kondisi khusus, serta pembuatan pseudocode dan flowchart untuk keenam program.

3. Implementasi program

Penulisan *source code* secara *modular* menggunakan bahasa R dan Python (total 12 *file* terpisah) di mana alur logika dari kedua bahasa tersebut harus setara. Implementasi ini juga mencakup kewajiban penggunaan fungsi (modularisasi) khusus untuk soal nomor 5 dan 6.

4. Pengujian dan dokumentasi

Melakukan tiga skenario uji untuk setiap program (mencakup input data normal dan tidak valid), seta mengertakan bukti hasil *running*, serta menyusun tabel *line-by-line code* untuk menjelaskan peran spesifik dari tiap baris instruksi yang ditulis.

BAB II

ANALISIS MASALAH

2.1 Soal *Project*

Bagian ini memuat rincian deskripsi soal proyek yang harus diselesaikan untuk masing-masing program.

2.1.1 Program 1

Buatlah sebuah program untuk menghitung jumlah kata dan jumlah kalimat dari sebuah teks. Asumsikan bahwa teks tersebut tidak memuat tanda titik selain untuk mengakhiri kalimat

2.1.2 Program 2

Buatlah objek string K1 sampai dengan K4 berikut dalam Python dan R sedemikian rupa sehingga ketika dicetak akan menampilkan kalimat-kalimat berikut secara tepat.

```
> print(K1)
    Saya tak 'kan menyerah.
> print(K2)
    Ia berkata, "Aku menyayangimu."
> print(K3)
    "Coba jelaskan pengertian 'cross-validation' dalam Machine
Learning!"
> print(K4)
    Surat keputusan itu bernomor 62/UN.34/19/2023.
```

2.1.3 Program 3

Akar-akar suatu persamaan kuadrat $ax^2 + bx + c = 0$ dapat ditentukan menggunakan rumus berikut

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

dan dapat bertipe bilangan real maupun bilangan imajiner. Buatlah sebuah program untuk membaca koefisien-koefisien persamaan kuadrat lalu menyajikan akar-akar real persamaan tersebut (bila ada) atau menyebutkan bahwa persamaan tersebut hanya memiliki akar-akar imajiner. Atur tampilan agar akar-akar bilangan real ditampilkan dalam tiga desimal.

2.1.4 Program 4

Delapan digit pertama pada NIP seorang ASN berturut-turut menyatakan tahun lahir, bulan lahir, dan tanggal lahir ASN tersebut. Sebagai contoh, seorang ASN dengan NIP 199301212019031010 lahir pada tahun 1993, bulan 1 (Januari), tanggal 21. Buatlah sebuah program untuk menampilkan tanggal lahir ASN bila diberikan NIP ASN tersebut. Gunakan percabangan untuk memilih nama bulan yang sesuai.

2.1.5 Program 5

Misal titik-titik A(2,1,3), B(1,-4,6), dan C(-2, 3, -2) ditetapkan sebagai pusat dari tiga cluster pada ruang dimensi tiga. Sembarang titik U(x1, x2, x3) digolongkan sebagai salah satu dari cluster A, B, atau C tergantung dari jarak terdekatnya dengan pusat masing-masing cluster; titik yang lebih dekat ke cluster A dibandingkan ke cluster B atau cluster C digolongkan sebagai anggota cluster A, demikian pula sebaliknya. Bila diketahui rumus jarak antara dua titik berikut:

$$jarak = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2}$$

Diberikan koordinat sembarang titik U yang dinyatakan sebagai tuple dengan tiga elemen. Buatlah program untuk menentukan apakah titik tersebut tergolong cluster A, B, atau C.

2.1.6 Program 6

Interval konfidensi $(1 - \alpha)$ untuk proporsi populasi p dapat dihitung sebagai

$$\hat{p} - z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} < p < \hat{p} + z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

dengan z bernilai 1,645 untuk α sebesar 10% dan 1,96 untuk α sebesar 5%. Buatlah sebuah program yang menerima masukan proporsi sampel $\hat{p}$ serta ukuran kepercayaan α dan menyajikan nilai interval konfidensi yang sesuai. Program harus menampilkan pesan kesalahan bila proporsi yang dimasukkan bernilai < 0 atau > 1 .

2.2 Tabel Input-Proses-Output

Tabel Input-Proses-Output (IPO) adalah bentuk ringkasan sederhana untuk memetakan alur kerja suatu program. Tabel ini digunakan untuk menjabarkan secara jelas apa saja data yang dibutuhkan (input), bagaimana data tersebut diolah (proses), serta hasil akhir yang akan ditampilkan (output), sehingga pembaca dapat memahami cara kerja program dengan mudah. Berikut adalah tabel Input-Proses-Output untuk program-program yang akan dirancang:

No.	Input	Proses	Output
1	Teks atau paragraf dalam bentuk data <i>string</i> yang dimasukkan langsung oleh pengguna melalui terminal atau konsol.	1. Mengambil teks yang dimasukkan pengguna dan menyimpannya ke dalam variabel teks. 2. Memisahkan <i>string</i> berdasarkan karakter spasi menggunakan fungsi <code>split()</code>, kemudian menghitung jumlah elemen atau panjang <i>array</i>/vektor yang dihasilkan menggunakan fungsi <code>len()</code> di Python atau <code>length()</code> di R. 3. Memisahkan <i>string</i> teks berdasarkan tanda titik (.) menggunakan fungsi pemisahan <i>string</i>, lalu menghitung total potongannya dan dikurangi 1 untuk mendapatkan jumlah kalimat yang valid.	Teks informasi yang menampilkan hasil perhitungan numerik berupa jumlah kata dan jumlah kalimat yang terkandung di dalam teks tersebut.
2	Tidak ada input yang dimasukkan oleh pengguna saat program berjalan. Nilai untuk variabel <i>string</i> dimasukkan secara langsung oleh penulis ke dalam kode.	1. Melakukan inisialisasi nilai secara langsung ke dalam empat variabel <i>string</i>, yaitu K1, K2, K3, dan K4. 2. Menangani karakter khusus pada masing-masing variabel menggunakan kombinasi tanda petik tunggal ('...'), tanda petik ganda ("..."), dan <i>escape character</i> (\) agar dapat membaca format <i>string</i> secara utuh tanpa terjadi <i>syntax error</i>.	Menampilkan isi dari variabel K1, K2, K3, dan K4 ke layar secara berurutan sehingga membentuk empat kalimat yang format <i>string</i> -nya sesuai dengan permintaan soal.
3	Tiga nilai numerik yang merepresentasikan koefisien persamaan kuadrat, yaitu nilai a, b, dan c (bertipe <i>integer</i> atau <i>float</i>) yang dimasukkan oleh pengguna.	1. Mengambil nilai koefisien a, b, dan c dari pengguna. 2. Menghitung nilai diskriminan (<i>D</i>) menggunakan rumus matematika: 3. Menggunakan struktur kondisi (if-else) untuk memeriksa nilai diskriminan: a. Jika $D \geq 0$, maka persamaan memiliki akar real. Program akan menghitung nilai akar menggunakan rumus ABC: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$	Tampilan nilai akar-akar real (x_1 dan x_2) dalam format tiga angka desimal, atau sebuah pesan tekstual yang menginformasikan bahwa persamaan kuadrat tersebut hanya memiliki akar-akar imajiner.

		b. Jika $D < 0$, program mengidentifikasi bahwa akar bersifat imajiner. 4. Mengatur format tampilan hasil akhir khusus untuk akar bilangan real agar dibulatkan secara presisi hingga 3 angka di belakang desimal.	
4	Deretan angka NIP ASN (18 digit) dari pengguna yang diterima dalam format <i>string</i> .	1. Menginisialisasi kamus data menggunakan tipe data <i>array</i> untuk menyimpan urutan nama bulan (Januari hingga Desember) dan batas maksimal jumlah hari setiap bulannya. 2. Mengekstrak 8 karakter pertama dari <i>string</i> NIP ASN, dengan rincian 4 digit awal untuk tahun, 2 digit untuk bulan, dan 2 karakter untuk tanggal lahir. 3. Mengonversi hasil ekstraksi teks tersebut menjadi tipe data numerik atau angka bulat agar dapat dilakukan perhitungan matematis. 4. Memvalidasi angka bulan (harus di antara 1 hingga 12) dan angka tanggal (d disesuaikan dengan batas jumlah hari per bulan, termasuk pengecekan kondisi tahun kabisat untuk bulan Februari). Jika tidak sesuai dengan batas-batas tersebut, program akan meminta pengguna untuk memasukkan ulang NIP. 5. Mengubah angka bulan yang sudah divalidasi menjadi nama bulan dalam bentuk teks (misalnya 1 menjadi "Januari") menggunakan acuan <i>array</i> yang telah dibuat beserta logika percabangan.	Menampilkan hasil ekstraksi, yaitu informasi tanggal lahir ASN ke layar secara lengkap yang mencakup tanggal, nama bulan, dan tahun lahir.
5	Koordinat titik $U(x_1, x_2, x_3)$. Input dapat diterima sebagai 3 nilai numerik terpisah yang kemudian disimpan ke	1. Menetapkan koordinat pusat tiga cluster yang sudah ditentukan: a. Cluster A(2, 1, 3) b. Cluster B(1, -4, 6) c. Cluster C(-2, 3, -2) 2. Membuat fungsi khusus untuk menghitung jarak Euclidean antara titik	Teks/string yang menampilkan informasi hasil klasifikasi cluster untuk titik U, yaitu apakah titik tersebut tergolong ke dalam Cluster

	dalam struktur data <i>tuple</i> (di Python) atau <i>vector</i> (di R).	input U dengan setiap pusat cluster (A, B, dan C) menggunakan rumus: $jarak = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2}$ - Membandingkan hasil ketiga jarak tersebut (jarak_a, jarak_b, jarak_c) menggunakan struktur percabangan (if-elif-else) untuk mencari nilai jarak yang paling minimum (terdekat). - Menentukan kelompok cluster berdasarkan jarak terpendek yang ditemukan.	A, Cluster B, atau Cluster C.
6	Proporsi sampel ($\hat{p}$), tingkat kepercayaan (α), dan ukuran sampel (n)	- Mengecek apakah $0 \leq \hat{p} \leq 1$. Jika tidak, maka tampilkan pesan kesalahan. - Menentukan nilai z. Jika $\alpha=0,05$ nilai $z=1,96$. Jika $\alpha=0,1$ nilai $z=1,645$. - Menghitung batas bawah dan batas atas dengan menggunakan rumus : $\hat{p} \pm z \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$ - Menampilkan nilai interval konfidensi	Nilai interval konfidensi atau pesan kesalahan.

2.3 Batasan Masalah

Batasan masalah pada Program 1 difokuskan pada perhitungan jumlah kata dan jumlah kalimat dari sebuah teks yang dimasukkan oleh pengguna. Program secara ketat mengasumsikan bahwa tanda titik (.) di dalam teks hanya digunakan secara eksklusif sebagai penanda untuk mengakhiri suatu kalimat dan tidak memuat tanda titik untuk keperluan lain, seperti penulisan singkatan nama, gelar akademik, ataupun penulisan angka desimal. Karakter pemisah antar kata yang dikenali oleh program hanyalah karakter spasi putih (*whitespace*) standar, sehingga tanda baca lainnya seperti koma (,) atau tanda hubung tetap dianggap melekat pada kata tersebut dan tidak dihitung sebagai pemisah kata mandiri. Selain itu, program ini tidak melakukan analisis semantik, pemeriksaan tata bahasa, ataupun validasi struktur kalimat, melainkan murni melakukan perhitungan berbasis manipulasi *string* (*string splitting*) menggunakan struktur logika yang setara pada bahasa pemrograman Python dan R.

Batasan masalah pada Program 2 difokuskan pada penanganan nilai objek *string* yang sudah ditetapkan di dalam kode. Secara spesifik, format *string* yang disusun harus sedemikian rupa agar dapat menampilkan kalimat dengan tepat saat dicetak, termasuk penggunaan tanda pertik

tunggal dan ganda untuk membungkus kalimat, serta karakter khusus yang diperlukan. Selain itu, program ini dirancang tidak menerima input dari pengguna, tidak melakukan pengolahan data, dan hanya menampilkan isi dari objek *string* sesuai format menggunakan bahasa pemrograman Python maupun R.

Batasan masalah pada Program 3 difokuskan pada proses membaca koefisien, menghitung, serta menyajikan akar-akar dari suatu persamaan kuadrat $ax^2 + bx + c = 0$ berdasarkan rumus ABC. Program ini dirancang untuk mendeteksi jenis akar melalui nilai diskriminan, di mana hasil perhitungan untuk akar-akar yang bertipe bilangan real akan diformat dan ditampilkan secara presisi dalam tiga desimal di layar. Namun, untuk kondisi di mana persamaan hanya memiliki akar-akar imajiner ($D < 0$), program dibatasi hanya untuk menampilkan pernyataan informatif bahwa akar bersifat imajiner tanpa menghitung, memformat, ataupun menjabarkan nilai komponen imajiner (i) tersebut secara mendetail. Selain itu, program mengasumsikan bahwa pengguna selalu memasukkan komponen input berupa bilangan numerik yang valid dan program tidak dirancang untuk menangani kondisi khusus jika nilai koefisien $a = 0$ yang secara matematis akan menyebabkan pembagian dengan nol dan mengubah esensi persamaan menjadi linear. Proses komputasi dan pemformatan hasil ini diimplementasikan menggunakan fungsi serta operasi matematika dasar tanpa melibatkan visualisasi grafis kurva fungsi kuadrat pada Python maupun R.

Batasan masalah pada program 4 difokuskan pada proses ekstraksi data tanggal lahir dari Nomor Induk Pegawai (NIP) ASN. Karena format NIP pada dasarnya merupakan deretan angka, program mengasumsikan bahwa input yang diberikan pengguna hanya terdiri atas karakter numerik dan tidak mengandung huruf, spasi, maupun karakter khusus. Apabila input yang diberikan tidak sesuai dengan asumsi tersebut, hasil pengolahan program dapat menjadi tidak valid karena tidak tersedia mekanisme validasi untuk menangani kesalahan tersebut. Selain itu, karena pengolahan hanya difokuskan pada delapan digit pertama untuk memperoleh informasi tahun, bulan, dan tanggal lahir, input yang hanya terdiri atas delapan digit angka tetap dapat diproses dan dianggap valid. Program tidak memverifikasi panjang NIP yang seharusnya terdiri atas 18 digit maupun keaslian data NIP yang dimasukkan. Validasi hanya terbatas pada pemeriksaan rentang bulan, batas tanggal pada setiap bulan, serta penyesuaian jumlah hari pada bulan Februari untuk tahun kabisat. Angka bulan yang telah dinyatakan valid kemudian diubah menjadi nama bulan menggunakan *array* yang telah dibuat dan hasil tanggal lahir akan ditampilkan ke layar.

Batasan masalah pada Program 5 difokuskan pada pengelompokan suatu titik sembarang $U(x_1, x_2, x_3)$ ke dalam salah satu dari tiga cluster yang koordinat pusatnya telah ditetapkan

secara statis dan konstan di dalam kode, yaitu Cluster A(2,1,3), Cluster B(1,-4,6), dan Cluster C(-2,3,-2). Program mengasumsikan bahwa input yang diberikan pengguna terdiri atas tepat tiga elemen numerik dalam ruang dimensi tiga yang diimplementasikan menggunakan struktur data *tuple* pada Python dan *vector* pada R. Proses perhitungan jarak Euclidean wajib dilakukan secara modular dengan menerapkan fungsi (*function*) khusus untuk menghitung jarak antara titik U dengan setiap pusat cluster. Meskipun program dapat memproses berbagai rentang nilai numerik baik bilangan bulat maupun desimal, program tidak dirancang untuk menangani koordinat di luar ruang dimensi tiga. Selain itu, apabila terjadi kondisi khusus di mana jarak titik U terhadap dua atau lebih pusat cluster bernilai sama persis (*tie-break*), program akan menampilkan keterangan titik berada pada kondisi khusus. Hasil akhir yang ditampilkan pada layar hanya terbatas pada teks informasi mengenai nama cluster terdekat yang berhasil diidentifikasi tanpa menyajikan visualisasi grafis 3D dari posisi titik-titik tersebut.

Batasan masalah pada program 6 difokuskan pada nilai proporsi populasi tunggal berdasarkan rumus yang telah ditentukan, dengan input wajib berupa proporsi sampel $\hat{p}$, ukuran sampel n , dan tingkat signifikansi α . Validasi input diterapkan secara ketat pada nilai proporsi sampel ukuran, di mana program akan menampilkan pesan kesalahan jika nilai $\hat{p}$ yang dimasukkan kurang dari 0 atau lebih dari 1. Selain itu, penentuan nilai kritis $z_{\alpha/2}$ dibatasi hanya untuk dua kondisi, yaitu nilai $z = 1,645$ untuk $\alpha = 10\%$ dan $z = 1,96$ untuk $\alpha = 5\%$, sehingga program tidak mendukung perhitungan nilai kritis di luar kedua parameter tingkat signifikansi tersebut.

2.4 Pembagian Tugas

Pembagian tugas dilakukan dengan cara membagi setiap anggota kelompok untuk menganalisis dan mengerjakan 1 sampai 2 program per individu anggota kelompok, dengan rincian pembagian tugas:

1. Aurn Nathan Fauzia, menganalisis dan mengerjakan program 1 dan 3
2. Rizky Aulia Putri, menganalisis dan mengerjakan program 2 dan 4
3. Lanang Tegar Harsono, menganalisis dan mengerjakan program 5
4. Muhamad Aji Firmansyah, menganalisis dan mengerjakan program 6

BAB III

PSEUDOCODE DAN FLOWCHART

3.1 Pseudocode

Pseudocode adalah cara menulis algoritma atau rangkaian instruksi program dalam bentuk deskriptif yang sederhana, agar mudah dipahami manusia. Pseudocode tidak terikat aturan sintaksis baku suatu bahasa pemrograman, sehingga sering disebut “kode semu”. Pseudocode digunakan sebagai kerangka awal untuk memetakan alur logika dan struktur data yang digunakan, sebelum akhirnya ditulis dalam kode yang sesungguhnya, sehingga pembuat program dapat merancang logika program tanpa terhambat dengan detail sintaks.

3.1.1 Program 1

Berikut adalah pseudocode untuk program yang dirancang guna menghitung jumlah kalimat dan jumlah kata dari sebuah teks masukan berdasarkan karakter pemisah tertentu, yaitu tanda titik untuk memisahkan kalimat dan spasi untuk memisahkan kata:

```
PROGRAM Hitung_Kalimat_Dan_Kata

DEKLARASI
    teks          : STRING
    JumlahKalimat : INTEGER
    JumlahKata    : INTEGER

ALGORITMA
    READ(teks)

    JumlahKalimat ← HITUNG_KARAKTER(teks, '.')
    JumlahKata    ← HITUNG_ELEMEN(PISAHKAN(teks, ' '))

    WRITE("Teks tersebut memuat ",
          JumlahKalimat,
          " kalimat dan ",
          JumlahKata,
          " kata.")

END PROGRAM
```

3.1.2 Program 2

Berikut adalah pseudocode untuk program yang dirancang guna menginisialisasi dan menampilkan beberapa variabel *string* yang memuat kombinasi tanda kutip dan karakter khusus menggunakan bantuan *escape character*:


```

PROGRAM String_Kalimat

DEKLARASI
    K1 : STRING
    K2 : STRING
    K3 : STRING
    K4 : STRING

ALGORITMA
    K1 ← "Saya tak 'kan menyerah."
    K2 ← 'Ia berkata, "Aku menyayangimu."'
    K3 ← "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
    K4 ← "Surat keputusan itu bernomor 62/UN.34/19/2023."

    WRITE(K1)
    WRITE(K2)
    WRITE(K3)
    WRITE(K4)

END PROGRAM

```

3.1.3 Program 3

Pseudocode di bawah ini merepresentasikan alur logika program dalam menghitung akar-akar persamaan kuadrat berdasarkan nilai koefisien yang diinputkan pengguna. Algoritma ini juga memuat pengecekan nilai diskriminan (D) untuk menentukan apakah akar yang dihasilkan bernilai kembar, real, atau imajiner:

```

PROGRAM Hitung_Akar_Persamaan_Kuadrat

DEKLARASI
    a, b, c      : REAL
    D             : REAL
    x, x1, x2    : REAL

ALGORITMA
    READ(a, b, c)

    D ← (b * b) - (4 * a * c)

    IF D = 0 THEN
        x ← -b / (2 * a)
        WRITE("Nilai x kembar = ", FORMAT_DESIMAL(x, 3))
    ELSE
        IF D > 0 THEN
            x1 ← (-b + AKAR(D)) / (2 * a)
            x2 ← (-b - AKAR(D)) / (2 * a)

```

```

        WRITE("Akar real (3 desimal):")
        WRITE("x1 = ", FORMAT_DESIMAL(x1, 3))
        WRITE("x2 = ", FORMAT_DESIMAL(x2, 3))

    ELSE
        WRITE("Persamaan hanya memiliki akar imajiner")
    ENDIF
ENDIF

END PROGRAM

```

3.1.4 Program 4

Pseudocode berikut menjabarkan alur kerja program dalam mengekstrak data tahun, bulan, dan tanggal lahir dari 18 digit input NIP ASN. Program ini dilengkapi dengan perulangan dan percabangan kondisi untuk memvalidasi batas hari dan aturan tahun kabisat:

```

PROGRAM NIP_ASN

DEKLARASI
    NIP          : STRING
    tahun        : INTEGER
    bulan        : INTEGER
    tanggal      : INTEGER
    nama_bulan   : STRING
    max_hari     : INTEGER

    data_bulan   : ARRAY
    data_hari    : ARRAY

ALGORITMA
    data_bulan ← [1:"Januari", 2:"Februari", 3:"Maret",
                  4:"April", 5:"Mei", 6:"Juni",
                  7:"Juli", 8:"Agustus", 9:"September",
                  10:"Oktober", 11:"November", 12:"Desember"]

    data_hari ← [1:31, 2:28, 3:31, 4:30, 5:31, 6:30,
                 7:31, 8:31, 9:30, 10:31, 11:30, 12:31]

LABEL_INPUT:
    READ(NIP)

    tahun ← INT(NIP[1:4])
    bulan ← INT(NIP[5:6])
    tanggal ← INT(NIP[7:8])

    IF (bulan < 1) OR (bulan > 12) THEN
        WRITE("NIP tidak valid, harap masukkan ulang.")
        GOTO LABEL_INPUT
    ENDIF

```

```

nama_bulan ← data_bulan[bulan]
max_hari ← data_hari[bulan]

IF (bulan = 2) AND
  (tahun MOD 400 = 0 OR
   (tahun MOD 4 = 0 AND tahun MOD 100 ≠ 0)) THEN
  max_hari ← 29
ENDIF

IF (tanggal < 1) OR (tanggal > max_hari) THEN
  WRITE("NIP tidak valid, harap masukkan ulang.")
  GOTO LABEL_INPUT
ENDIF

WRITE("Tanggal lahir Anda: ",
      tanggal, " ",
      nama_bulan, " ",
      tahun)

END PROGRAM

```

3.1.5 Program 5

Berikut adalah pseudocode untuk program klasifikasi *cluster* dalam ruang tiga dimensi (3D). Program ini bekerja dengan cara menghitung jarak menggunakan formula *Euclidean* antara titik input pengguna dengan tiga titik pusat *cluster* tetap, kemudian mengklasifikasikannya ke dalam *cluster* dengan jarak terdekat:

```

PROGRAM KlasifikasiCluster3D
DEKLARASI
  x1, x2, x3 : REAL
  dA, dB, dC : REAL
ALGORITMA
  WRITE("=== Sistem Klasifikasi Cluster 3D ===")
  WRITE("Titik Pusat Cluster Tetap:")
  WRITE("A(2,1,3), B(1,-4,6), C(-2,3,-2)")

  WRITE("Masukkan koordinat U (x1, x2, x3):")
  READ(x1, x2, x3)

  IF x1 BUKAN_ANGKA OR
    x2 BUKAN_ANGKA OR
    x3 BUKAN_ANGKA THEN

    WRITE("Tampilkan Salah Input")

  ELSE

    dA ← SQRT((x1 - 2)^2 + (x2 - 1)^2 + (x3 - 3)^2)

```

```

dB ← SQRT((x1 - 1)^2 + (x2 - (-4))^2 + (x3 - 6)^2)
dC ← SQRT((x1 - (-2))^2 + (x2 - 3)^2 + (x3 - (-2))^2)

IF (dA = dB) < dC OR (dB = dC) < dA OR (dA = dC) < dB THEN

    WRITE("Tampilkan Kondisi Khusus / Nilai Jarak Sama")

ELSE IF (dA < dB AND dB < dC) OR (dA < dC AND dC < dB) THEN

    WRITE("Tampilkan Cluster A dan Nilai Jarak")

ELSE IF (dB < dC AND dC < dA) OR (dB < dA AND dA < dC) THEN

    WRITE("Tampilkan Cluster B dan Nilai Jarak")

ELSE

    WRITE("Tampilkan Cluster C dan Nilai Jarak")

ENDIF

ENDIF

END PROGRAM

```

3.1.6 Program 6

Pseudocode di bawah ini menguraikan langkah-langkah logis program dalam menghitung estimasi interval konfidensi untuk suatu proporsi. Algoritma ini mencakup validasi batas nilai input proporsi, pemilihan nilai Z berdasarkan tingkat signifikansi (α), serta perhitungan *margin of error*:

```

Program Menghitung Interval Konfidensi

DEKLARASI

p_hat, alpha, z : REAL

margin_of_error, batas_bawah, batas_atas : REAL

n : INTEGER

BEGIN

    Input p_hat, alpha, n

    //Validasi input proporsi

    IF (p_hat < 0 OR p_hat > 1) THEN

        PRINT("Input harus bernilai antara 0 dan 1")
    
```

```

EXIT
ENDIF
//Menentukan nilai z berdasarkan alpha
IF(alpha == 0.1) THEN
    z <- 1.645
ELSE IF (alpha == 0.05) THEN
    z <- 1.96
ELSE
    PRINT("Nilai alpha hanya mendukung 0.05 dan 0.1")
    EXIT
ENDIF
//Pehitungan
margin_of_error <- z * SQRT((p_hat * (1-p_hat))/n)
batas_bawah <- p_hat - margin_of_error
batas_atas <- p_hat + margin_of_error
//Output
Print("Interval konfidensi :", batas_bawah, "< p <", batas_atas)
END PROGRAM

```

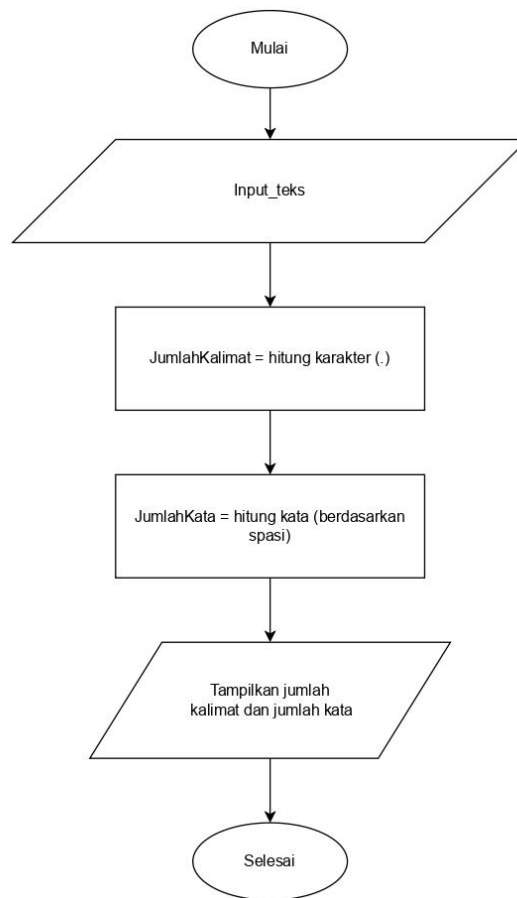
3.2 Flowchart

Flowchart (diagram alir) adalah cara memvisualisasikan algoritma atau rangkaian instruksi program dalam bentuk simbol-simbol geometri (persegi panjang, belah ketupat, oval, dan lain-lain), agar alur logika lebih mudah dipahami manusia, Flowchart memetakan urutan proses secara sistematis. Mulai dari tahap memasukkan data, proses, evaluasi kondisi (percabangan), hingga menampilkan hasil akhir. Diagram alir ini digunakan sebagai gambaran visual awal dari struktur jalannya program, sebelum akhirnya diimplementasikan ke dalam bahasa program sesungguhnya, sehingga pembuat program dapat merancang dan melihat aliran proses eksekusi program secara jelas.

3.2.1 Program 1

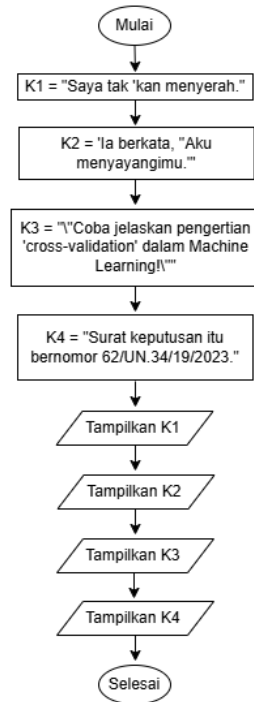
Flowchart ini memvisualisasikan proses pembacaan input teks dari pengguna, yang kemudian diproses untuk menghitung jumlah kalimat berdasarkan karakter pemisah titik, serta

menghitung jumlah kata berdasarkan karakter pemisah spasi, sebelum akhirnya menampilkan kedua hasil perhitungan tersebut ke layar.



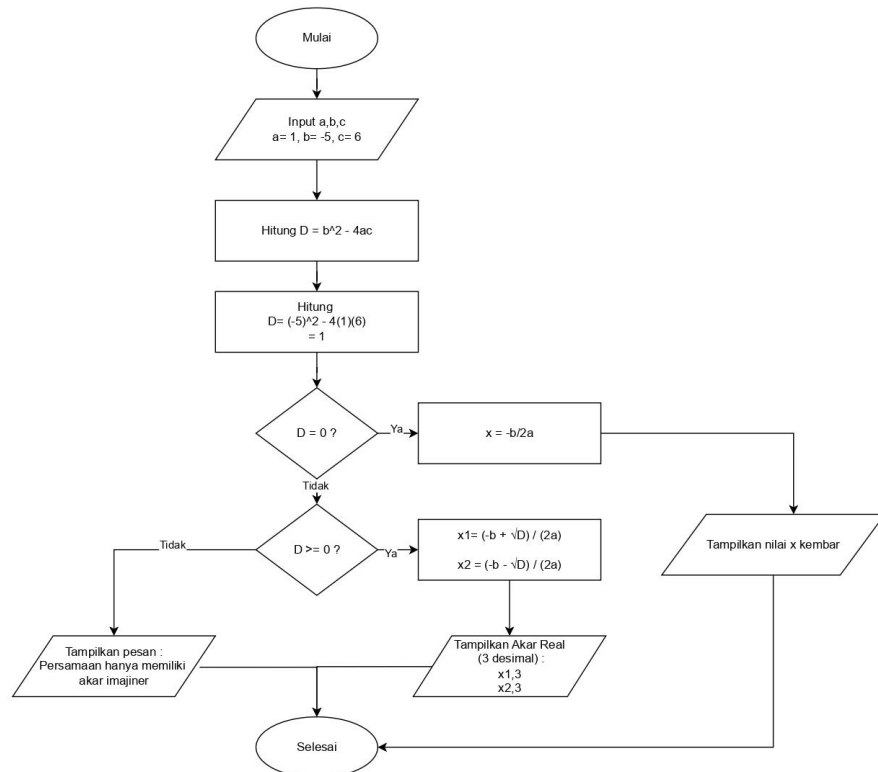
3.2.2 Program 2

Flowchart berikut menggambarkan alur program yang berjalan secara lurus (*sekuensial*) tanpa melibatkan input dari pengguna. Alur utamanya difokuskan pada proses inisialisasi empat variabel *string* yang memuat karakter khusus, dilanjutkan dengan instruksi pencetakan masing-masing variabel tersebut secara berurutan.



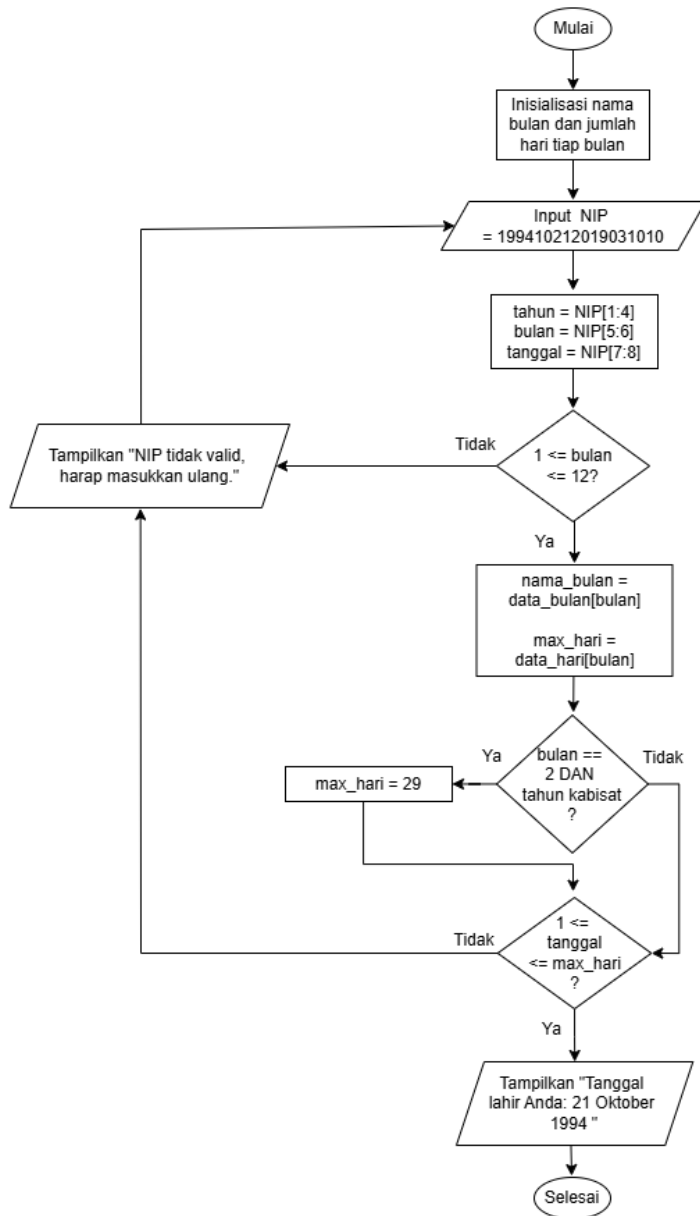
3.2.3 Program 3

Pada flowchart ini, alur program difokuskan pada penerimaan input nilai koefisien (a, b, dan c) yang dilanjutkan dengan operasi perhitungan nilai diskriminan (D). Diagram ini memuat percabangan kondisi bersarang untuk mengarahkan program dalam menentukan apakah akar persamaan tersebut bersifat kembar, real, atau imajiner.



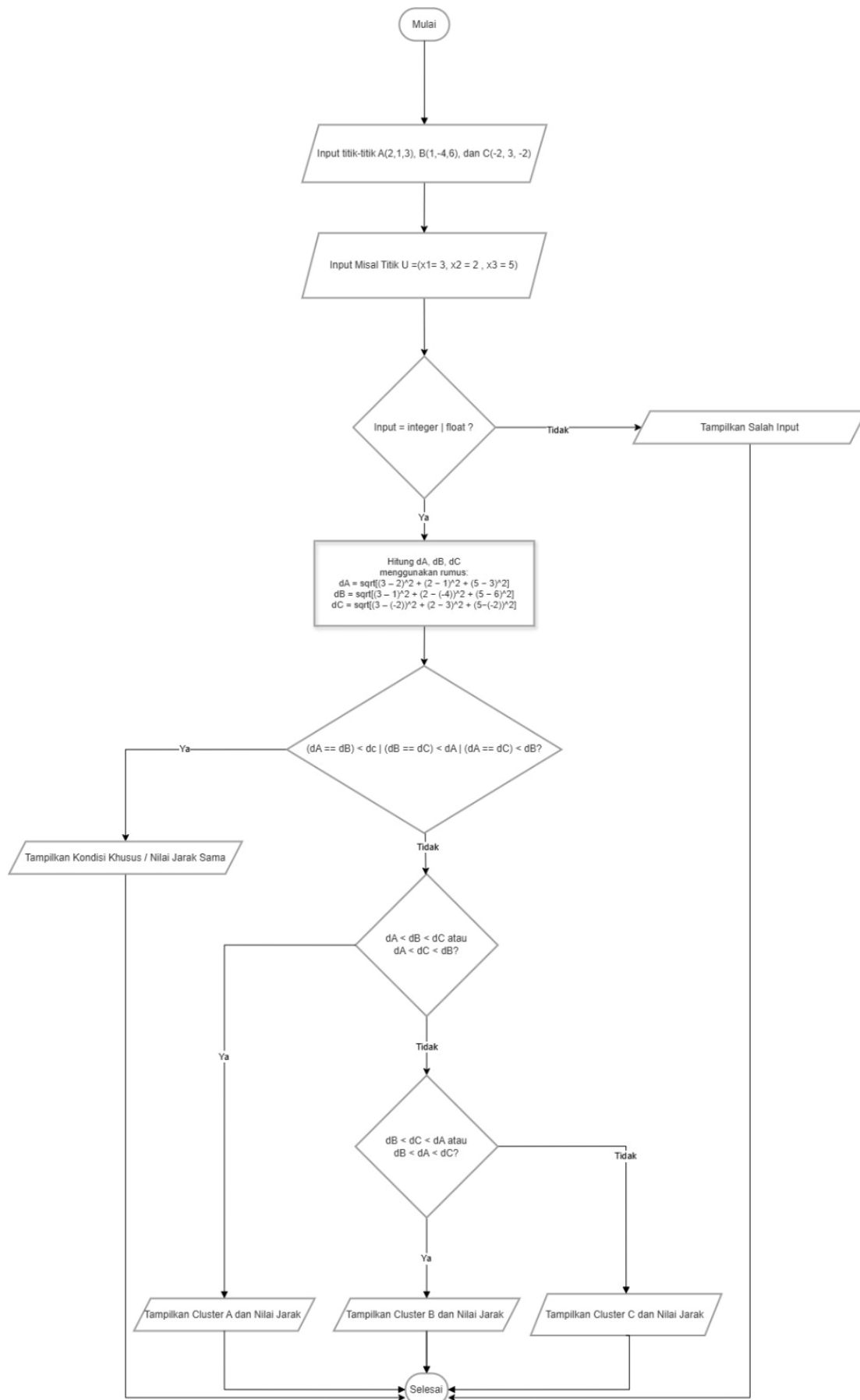
3.2.4 Program 4

Flowchart di bawah ini menunjukkan alur sistematis dalam mengekstrak data penanggalan dari 18 digit NIP. Diagram ini memuat struktur perulangan (*looping*) untuk meminta input ulang apabila data tidak rasional, serta percabangan kompleks yang mengevaluasi batas maksimal hari dan aturan perhitungan tahun kabisat.



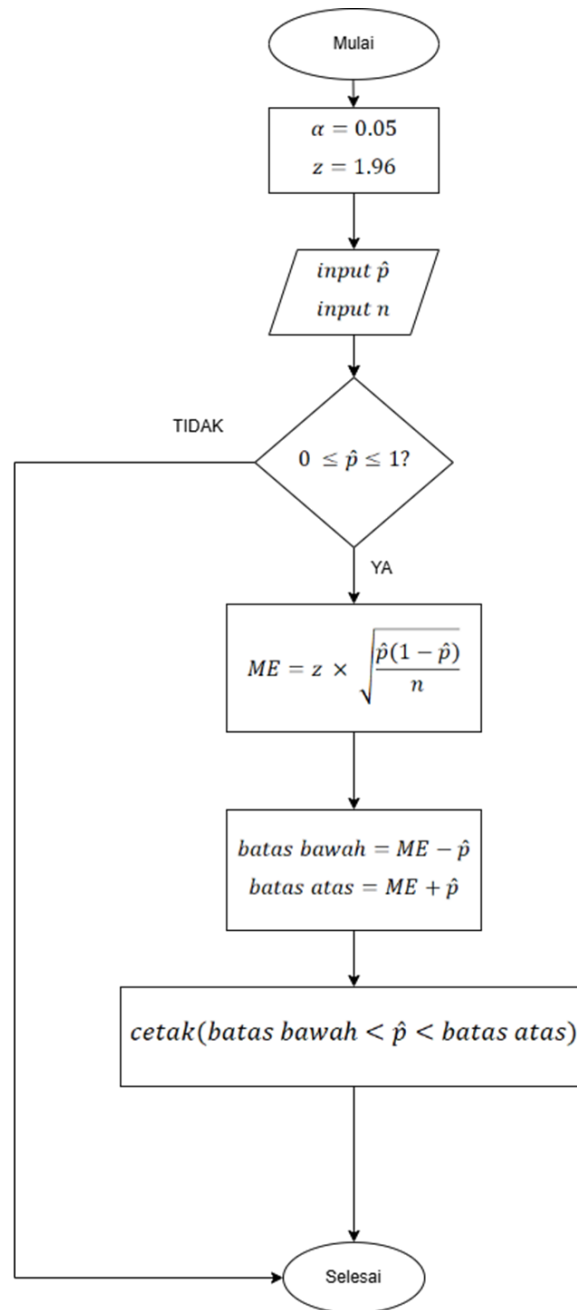
3.2.5 Program 5

Diagram alir ini merepresentasikan langkah-langkah program dalam menerima input koordinat titik spasial dan menghitung jaraknya terhadap tiga titik pusat *cluster* tetap menggunakan formula *Euclidean*. Alur kemudian bercabang untuk mengevaluasi dan membandingkan ketiga jarak tersebut guna menentukan hasil klasifikasi *cluster* terdekat.



3.2.6 Program 6

Flowchart berikut menggambarkan proses perhitungan interval konfidensi. Alur dimulai dari penerimaan data proporsi, tingkat signifikansi (α), dan ukuran sampel, dilanjutkan dengan percabangan untuk memvalidasi input, penentuan nilai probabilitas Z, perhitungan *margin of error*, hingga tahap akhir berupa pencetakan batas bawah dan batas atas interval.



BAB IV

IMPLEMENTASI PROGRAM

4.1 Program 1

Berikut adalah kode sumber (*source code*) untuk Program 1. Implementasi program ini dibagi ke dalam dua bahasa pemrograman, yaitu R dan Python.

4.1.1 Code R :

```
#Program 1
#Kondisi 1
teks <- "Ibu sedang masak di dapur, sedangkan ayah mencuci motor. Kucing
tidur di atas kasur, sementara burung bernyanyi di luar sangkar."
jumlahKalimat <- 0
for(i in 1:nchar(teks)){
  if(substr(teks, i, i) == "."){
    jumlahKalimat <- jumlahKalimat + 1
  }
}

kata <- strsplit(teks, " ")
jumlahKata <- length(kata[[1]])

cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata,
"kata")

## Teks tersebut memuat 2 kalimat dan 20 kata

#Kondisi 2
teks <- "Indonesia merdeka pada tanggal 17 Agustus 1945."

jumlahKalimat <- 0
for(i in 1:nchar(teks)){
  if(substr(teks, i, i) == "."){
    jumlahKalimat <- jumlahKalimat + 1
  }
}

kata <- strsplit(teks, " ")
jumlahKata <- length(kata[[1]])

cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata,
"kata")

## Teks tersebut memuat 1 kalimat dan 7 kata

#Kondisi 3
teks <- "Harga buku ini 50000 namun aku hanya membawa 30000."

jumlahKalimat <- 0
for(i in 1:nchar(teks)){
```

```

    if(substr(teks, i, i) == "."){
        jumlahKalimat <- jumlahKalimat + 1
    }
}

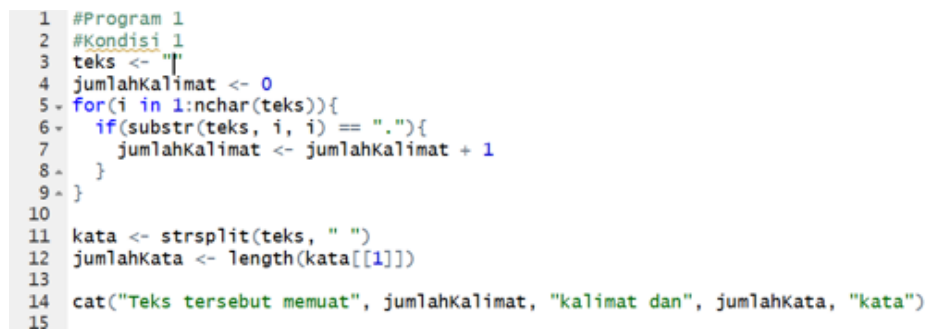
kata <- strsplit(teks, " ")
jumlahKata <- length(kata[[1]])

cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata,
    "kata")

## Teks tersebut memuat 1 kalimat dan 9 kata

```

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 1:



```

1 #Program 1
2 #Kondisi 1
3 teks <- ""
4 jumlahKalimat <- 0
5 for(i in 1:nchar(teks)){
6     if(substr(teks, i, i) == "."){
7         jumlahKalimat <- jumlahKalimat + 1
8     }
9 }
10
11 kata <- strsplit(teks, " ")
12 jumlahKata <- length(kata[[1]])
13
14 cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata, "kata")
15

```

4.1.2 Code Python :

```

#Program 1
#Kondisi 1
teks = input("Masukkan Teks:")
#Hitung Kalimat
jumlah_kalimat = 0
for karakter in teks:
    if karakter == ".":
        jumlah_kalimat += 1
#Hitung Kata
jumlah_kata = 1
for karakter in teks:
    if karakter == " ":
        jumlah_kata += 1

print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata,
    "kata.")

#Program 1
#Kondisi 2
teks = input("Masukkan Teks:")

#Hitung Kalimat
jumlah_kalimat = 0

```

```

for karakter in teks:
    if karakter == "." :
        jumlah_kalimat += 1
#Hitung Kata
jumlah_kata = 1
for karakter in teks :
    if karakter == " ":
        jumlah_kata += 1

print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata,
"kata.")

#Program 1
#Kondisi 3
teks = input("Masukkan Teks:")

#Hitung Kalimat
jumlah_kalimat = 0
for karakter in teks:
    if karakter == "." :
        jumlah_kalimat += 1
#Hitung Kata
jumlah_kata = 1
for karakter in teks :
    if karakter == " ":
        jumlah_kata += 1

print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata,
"kata.")

Masukkan Teks:Ibu sedang masak di dapur, sedangkan ayah mencuci motor.
Kucing tidur di atas kasur, sementara burung bernyanyi di luar sangkar.
Teks tersebut memuat 2 kalimat dan 20 kata.
Masukkan Teks:Indonesia merdeka pada tanggal 17 Agustus 1945.
Teks tersebut memuat 1 kalimat dan 7 kata.
Masukkan Teks:Harga buku ini 50000 namun aku hanya membawa 30000.
Teks tersebut memuat 1 kalimat dan 9 kata.

```

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 1:

```

#Program 1
teks = input("Masukkan teks: ")

# Hitung kalimat
jumlah_kalimat = 0
for karakter in teks:
    if karakter == ".":
        jumlah_kalimat += 1

# Hitung kata
jumlah_kata = 1
for karakter in teks:
    if karakter == " ":
        jumlah_kata += 1

print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.")

```

4.2 Program 2

Berikut adalah kode sumber (*source code*) untuk Program 2. Implementasi program ini dibagi ke dalam dua bahasa pemrograman, yaitu R dan Python.

4.2.1 Code R :

```
#Program String_Kalimat
K1 <- "Saya tak 'kan menyerah."
K2 <- 'Ia berkata, "Aku menyayangimu."'
K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."
cat(K1, "\n")
cat(K2, "\n")
cat(K3, "\n")
cat(K4, "\n")
```

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 2:

```
K1 <- "Saya tak 'kan menyerah."
K2 <- 'Ia berkata, "Aku menyayangimu."'
K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."
```

4.2.2 Code Python

```
#Program String_Kalimat
K1 = "Saya tak 'kan menyerah."
K2 = 'Ia berkata, "Aku menyayangimu."'
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."
print(K1)
print(K2)
print(K3)
print(K4)
```

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 2:

```
K1 = "Saya tak 'kan menyerah."
K2 = 'Ia berkata, "Aku menyayangimu."'
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."
```

4.3 Program 3

Berikut adalah kode sumber (*source code*) untuk Program 3. Implementasi program ini dibagi ke dalam dua bahasa pemrograman, yaitu R dan Python.

4.3.1 Code R

```
#Program 3
#Kondisi 1
a <- 1
b <- 2

c <- 1

D <- b^2 - 4*a*c

if (D == 0) {

  x <- -b / (2*a)

  cat("Nilai x kembar =", round(x, 3), "\n")

} else if (D > 0) {

  x1 <- (-b + sqrt(D)) / (2*a)
  x2 <- (-b - sqrt(D)) / (2*a)

  cat("Akar real:\n")
  cat("x1 =", round(x1, 3), "\n")
  cat("x2 =", round(x2, 3), "\n")

} else {

  cat("Persamaan hanya memiliki akar imajiner\n")

}

## Nilai x kembar = -1

#Kondisi 2
a <- 1
b <- -5
c <- 6

D <- b^2 - 4*a*c

if (D == 0) {

  x <- -b / (2*a)
  cat("Nilai x kembar =", round(x, 3), "\n")

} else if (D > 0) {

  x1 <- (-b + sqrt(D)) / (2*a)
  x2 <- (-b - sqrt(D)) / (2*a)

  cat("Akar real:\n")
  cat("x1 =", round(x1, 3), "\n")
  cat("x2 =", round(x2, 3), "\n")

}
```

```

} else {

  cat("Persamaan hanya memiliki akar imajiner\n")

}

## Akar real:
## x1 = 3
## x2 = 2

#Kondisi 3
a <- 1
b <- 2
c <- 5

D <- b^2 - 4*a*c

if (D == 0) {

  x <- -b / (2*a)

  cat("Nilai x kembar =", round(x, 3), "\n")

} else if (D > 0) {

  x1 <- (-b + sqrt(D)) / (2*a)
  x2 <- (-b - sqrt(D)) / (2*a)

  cat("Akar real:\n")
  cat("x1 =", round(x1, 3), "\n")
  cat("x2 =", round(x2, 3), "\n")

} else {

  cat("Persamaan hanya memiliki akar imajiner\n")

}

## Persamaan hanya memiliki akar imajiner

```


Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 3:

```
1 #Program 3
2 a <-
3 b <-
4 c <-
5
6 D <- b^2 - 4*a*c
7
8 if (D == 0) {
9
10 x <- -b / (2*a)
11
12 cat("Nilai x kembar =", round(x, 3), "\n")
13
14 } else if (D > 0) {
15
16 x1 <- (-b + sqrt(D)) / (2*a)
17 x2 <- (-b - sqrt(D)) / (2*a)
18
19 cat("Akar real:\n")
20 cat("x1 =", round(x1, 3), "\n")
21 cat("x2 =", round(x2, 3), "\n")
22
23 } else {
```

4.3.2 Code Python

```
# Program 3
# Menghitung akar-akar persamaan kuadrat

a = float(input("Masukkan nilai a: "))
b = float(input("Masukkan nilai b: "))
c = float(input("Masukkan nilai c: "))

D = b**2 - 4*a*c

if D == 0:
    x = -b / (2 * a)
    print("Nilai x kembar =", round(x, 3))
elif D > 0:
    x1 = (-b + D**0.5) / (2 * a)
    x2 = (-b - D**0.5) / (2 * a)
    print("Akar real:")
    print("x1 =", round(x1, 3))
    print("x2 =", round(x2, 3))
else:
    print("Persamaan hanya memiliki akar imajiner")

Kondisi #1

Masukkan nilai a: 1
Masukkan nilai b: 2
Masukkan nilai c: 1

Kondisi #2

Masukkan nilai a: 1
Masukkan nilai b: -5
Masukkan nilai c: 6
```

Kondisi #3

Masukkan nilai a: 1
Masukkan nilai b: 2
Masukkan nilai c: 5

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 3:

```
a = float(input("Masukkan nilai a: "))
b = float(input("Masukkan nilai b: "))
c = float(input("Masukkan nilai c: "))

D = b**2 - 4*a*c

if D == 0:
    x = -b / (2 * a)
    print("Nilai x kembar =", round(x, 3))

elif D > 0:
    x1 = (-b + D**0.5) / (2 * a)
    x2 = (-b - D**0.5) / (2 * a)

    print("Akar real:")
    print("x1 =", round(x1, 3))
    print("x2 =", round(x2, 3))

else:
    print("Persamaan hanya memiliki akar imajiner")
```

4.4 Program 4

Berikut adalah kode sumber (*source code*) untuk Program 4. Implementasi program ini dibagi ke dalam dua bahasa pemrograman, yaitu R dan Python.

4.4.1 Code R

```
#Program NIP_ASN
#Inisialisasi kamus bulan dan batas hari
data_bulan <- c("Januari", "Februari", "Maret", "April", "Mei", "Juni",
                "Juli", "Agustus", "September", "Oktober", "November", "Desember")
data_hari <- c(31, 28, 31, 30, 31, 30,
               31, 31, 30, 31, 30, 31)
while (TRUE) {
  #Menerima input dari user
  NIP <- readline(prompt = "Masukkan 18 digit NIP Anda: ")
  tahun <- as.numeric(substr(NIP, 1, 4)) #Indeks R mulai dari 1
  bulan <- as.numeric(substr(NIP, 5, 6))
  tanggal <- as.numeric(substr(NIP, 7, 8))
```

```

#Validasi rentang bulan
if (bulan < 1 | bulan > 12){
  cat("NIP tidak valid, harap masukkan ulang.\n\n")
  next
}

#Mengambil data dari kamus
nama_bulan <- data_bulan[bulan]
max_hari <- data_hari[bulan]

#Cek tahun kabisat (update batas Februari)
if (bulan == 2 & (tahun %% 400 == 0 | (tahun %% 4 == 0 & tahun %% 100 != 0))){
  max_hari = 29
}

#Validasi batas tanggal
if (tanggal <1 | tanggal > max_hari){
  cat("NIP tidak valid, harap masukkan ulang.\n\n")
  next
}

#Jika semua valid
cat("Tanggal lahir Anda:", tanggal, nama_bulan, tahun, "\n")
break
}

```

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 4:

```

while (TRUE) {
  #Menerima input dari user
  NIP <- readline(prompt = "Masukkan 18 digit NIP Anda: ")
  tahun <- as.numeric(substr(NIP, 1, 4)) #Indeks R mulai dari 1
  bulan <- as.numeric(substr(NIP, 5, 6))
  tanggal <- as.numeric(substr(NIP, 7, 8))

  #Validasi rentang bulan
  if (bulan < 1 | bulan > 12){
    cat("NIP tidak valid, harap masukkan ulang.\n\n")
    next
  }

  #Mengambil data dari kamus
  nama_bulan <- data_bulan[bulan]
  max_hari <- data_hari[bulan]

  #Cek tahun kabisat (update batas Februari)
  if (bulan == 2 & (tahun %% 400 == 0 | (tahun %% 4 == 0 & tahun %% 100 != 0))){
    max_hari = 29
  }

  #Validasi batas tanggal
  if (tanggal <1 | tanggal > max_hari){
    cat("NIP tidak valid, harap masukkan ulang.\n\n")
    next
  }

  #Jika semua valid
  cat("Tanggal lahir Anda:", tanggal, nama_bulan, tahun, "\n")
  break
}

```

4.4.2 Code Python

```
#Program NIP_ASN
#Inisialisasi kamus bulan dan batas hari
data_bulan = {
    1:"Januari", 2:"Februari", 3:"Maret", 4:"April", 5:"Mei", 6:"Juni",
    7:"Juli", 8:"Agustus", 9:"September", 10:"Oktober", 11:"November",
    12:"Desember"
}
data_hari = {
    1:31, 2:28, 3:31, 4:30, 5:31, 6:30,
    7:31, 8:31, 9:30, 10:31, 11:30,
    12:31
}

while True:
    NIP = input("Masukkan 18 digit NIP Anda:")
    tahun = int(NIP[0:4]) #Indeks Python mulai dari 0
    bulan = int(NIP[4:6])
    tanggal = int(NIP[6:8])

    #Validasi rentang bulan
    if bulan < 1 or bulan > 12:
        print("NIP tidak valid, harap masukkan ulang.")
        continue

    #Mengambil data dari kamus
    nama_bulan = data_bulan[bulan]
    max_hari = data_hari[bulan]

    #Cek tahun kabisat (update batas Februari)
    if bulan == 2 and (tahun % 400 == 0 or (tahun % 4 == 0 and tahun % 100
    != 0)):
        max_hari = 29

    #Validasi batas tanggal
    if tanggal <1 or tanggal > max_hari:
        print("NIP tidak valid, harap masukkan ulang.")
        continue

    #Jika semua valid
    print("Tanggal lahir Anda:", tanggal, nama_bulan, tahun)
    break
```

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 4:

```
while True:
    NIP = input("Masukkan 18 digit NIP Anda:")
    tahun = int(NIP[0:4]) #Indeks Python mulai dari 0
    bulan = int(NIP[4:6])
    tanggal = int(NIP[6:8])

    #Validasi rentang bulan
    if bulan < 1 or bulan > 12:
        print("NIP tidak valid, harap masukkan ulang.")
        continue

    #Mengambil data dari kamus
    nama_bulan = data_bulan[bulan]
    max_hari = data_hari[bulan]

    #Cek tahun kabisat (update batas Februari)
    if bulan == 2 and (tahun % 400 == 0 or (tahun % 4 == 0 and tahun % 100 != 0)):
        max_hari = 29

    #Validasi batas tanggal
    if tanggal <1 or tanggal > max_hari:
        print("NIP tidak valid, harap masukkan ulang.")
        continue

    #Jika semua valid
    print("Tanggal lahir Anda:", tanggal, nama_bulan, tahun)
    break
```

4.5 Program 5

Berikut adalah kode sumber (*source code*) untuk Program 5. Implementasi program ini dibagi ke dalam dua bahasa pemrograman, yaitu R dan Python.

4.5.1 Code R

```
tentukan_cluster <- function(x1, x2, x3) {

    # KONDISI 2: Validasi Error Input (Bukan Angka)
    if (!is.numeric(x1) || !is.numeric(x2) || !is.numeric(x3)) {
        cat(sprintf("Menguji Titik U: (%s, %s, %s)\n", as.character(x1),
as.character(x2), as.character(x3)))
        cat("Hasil: KONDISI ERROR (Input harus berupa angka!)\n\n")
    }else{

    # Tetapkan koordinat pusat cluster tetap
    A <- c(2, 1, 3)
    B <- c(1, -4, 6)
    C <- c(-2, 3, -2)
```

```

# Gabungkan parameter menjadi vektor U
U <- c(x1, x2, x3)

# Fungsi internal untuk menghitung jarak Euclidean
hitung_jarak <- function(titik1, titik2) {
  return(sqrt(sum((titik1 - titik2)^2)))
}

# Hitung jarak dari U ke setiap cluster
jarak_A <- hitung_jarak(U, A)
jarak_B <- hitung_jarak(U, B)
jarak_C <- hitung_jarak(U, C)

# Cetak informasi titik dan jarak
cat(sprintf("Menguji Titik U: (%.1f, %.1f, %.1f)\n", x1, x2, x3))
cat(sprintf("- Jarak ke Cluster A: %.4f\n", jarak_A))
cat(sprintf("- Jarak ke Cluster B: %.4f\n", jarak_B))
cat(sprintf("- Jarak ke Cluster C: %.4f\n", jarak_C))

# KONDISI 1: Kondisi Khusus (Deteksi Cluster yang Jaraknya Sama)
if (jarak_A == jarak_B && jarak_B == jarak_C) {
  cat("Hasil: KONDISI KHUSUS (Titik U berada tepat di tengah-tengah
antara Cluster A, B, dan C)\n\n")
} else if (jarak_A == jarak_B && jarak_A <= jarak_C) {
  cat("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang
SAMA antara CLUSTER A dan CLUSTER B)\n\n")
} else if (jarak_A == jarak_C && jarak_A <= jarak_B) {
  cat("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang
SAMA antara CLUSTER A dan CLUSTER C)\n\n")
} else if (jarak_B == jarak_C && jarak_B <= jarak_A) {
  cat("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang
SAMA antara CLUSTER B dan CLUSTER C)\n\n")
}

# KONDISI 3: Kondisi Normal (Satu Cluster Terdekat)
else if (jarak_A < jarak_B && jarak_A < jarak_C) {
  cat("Hasil: KONDISI NORMAL (CLUSTER A)\n\n")
} else if (jarak_B < jarak_C) {
  cat("Hasil: KONDISI NORMAL (CLUSTER B)\n\n")
} else {
  cat("Hasil: KONDISI NORMAL (CLUSTER C)\n\n")
}
}
}

# UJI COBA KONDISI
tentukan_cluster(0, 2, 0.5)

## Menguji Titik U: (0.0, 2.0, 0.5)
## - Jarak ke Cluster A: 3.3541
## - Jarak ke Cluster B: 8.2006
## - Jarak ke Cluster C: 3.3541

```

```
## Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA  
antara CLUSTER A dan CLUSTER C)
```

```
tentukan_cluster(-1, 2, -1)
```

```
## Menguji Titik U: (-1.0, 2.0, -1.0)
```

```
## - Jarak ke Cluster A: 5.0990
```

```
## - Jarak ke Cluster B: 9.4340
```

```
## - Jarak ke Cluster C: 1.7321
```

```
## Hasil: KONDISI NORMAL (CLUSTER C)
```

```
tentukan_cluster("Lanang", 3, 5)
```

```
## Menguji Titik U: (Lanang, 3, 5)
```

```
## Hasil: KONDISI ERROR (Input harus berupa angka!)
```

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 5:

```
34 # KONDISI 2: Kondisi Khusus (Deteksi Cluster yang Jaraknya Sama)
35 if (jarak_A == jarak_B && jarak_B == jarak_C) {
36   cat("Hasil: KONDISI KHUSUS (Titik U berada tepat di tengah-tengah antara Cluster A, B, dan C)\n\n")
37 } else if (jarak_A == jarak_B && jarak_A <= jarak_C) {
38   cat("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER B)\n\n")
39 } else if (jarak_A == jarak_C && jarak_A <= jarak_B) {
40   cat("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER C)\n\n")
41 } else if (jarak_B == jarak_C && jarak_B <= jarak_A) {
42   cat("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER B dan CLUSTER C)\n\n")
43 }
44
45 # KONDISI 3: Kondisi Normal (Satu Cluster Terdekat)
46 else if (jarak_A < jarak_B && jarak_A < jarak_C) {
47   cat("Hasil: KONDISI NORMAL (CLUSTER A)\n\n")
48 } else if (jarak_B < jarak_C) {
49   cat("Hasil: KONDISI NORMAL (CLUSTER B)\n\n")
50 } else {
51   cat("Hasil: KONDISI NORMAL (CLUSTER C)\n\n")
52 }
```

4.5.2 Code Python :

```
import math

def tentukan_cluster(x1, x2, x3):

    # KONDISI 1: Validasi Error Input (Bukan Angka)
    # Memeriksa apakah ada parameter yang bukan int atau float
    if not (isinstance(x1, (int, float)) and
            isinstance(x2, (int, float)) and
            isinstance(x3, (int, float))):
        print(f"Menguji Titik U: ({x1}, {x2}, {x3})")
        print("Hasil: KONDISI ERROR (Input harus berupa angka!)\n")
        return # Keluar dari fungsi dengan aman

    # Tetapkan koordinat pusat cluster tetap
    A = (2, 1, 3)
    B = (1, -4, 6)
    C = (-2, 3, -2)

    # Fungsi internal untuk menghitung jarak Euclidean
    def hitung_jarak(titik1, titik2):
        return math.sqrt((titik1[0] - titik2[0])**2 +
                          (titik1[1] - titik2[1])**2 +
```

```

        (titik1[2] - titik2[2])**2)

# Hitung jarak dari U ke setiap cluster
U = (x1, x2, x3)
jarak_A = hitung_jarak(U, A)
jarak_B = hitung_jarak(U, B)
jarak_C = hitung_jarak(U, C)

# Cetak informasi titik dan jarak (Gunakan format .7f agar presisi
desimalnya terlihat)
print(f"Menguji Titik U: ({x1:.7f}, {x2:.7f}, {x3:.7f})")
print(f"- Jarak ke Cluster A: {jarak_A:.4f}")
print(f"- Jarak ke Cluster B: {jarak_B:.4f}")
print(f"- Jarak ke Cluster C: {jarak_C:.4f}")

# KONDISI 2: Kondisi Khusus (Deteksi Cluster yang Jaraknya Sama)

jA, jB, jC = round(jarak_A, 7), round(jarak_B, 7), round(jarak_C, 7)

if jA == jB and jB == jC:
    print("Hasil: KONDISI KHUSUS (Titik U berada tepat di tengah-
tengah antara Cluster A, B, dan C)\n")
    elif jA == jB and jA <= jC:
        print("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat
yang SAMA antara CLUSTER A dan CLUSTER B)\n")
    elif jA == jC and jA <= jB:
        print("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat
yang SAMA antara CLUSTER A dan CLUSTER C)\n")
    elif jB == jC and jB <= jA:
        print("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat
yang SAMA antara CLUSTER B dan CLUSTER C)\n")

# KONDISI 3: Kondisi Normal (Satu Cluster Terdekat)
elif jA < jB and jA < jC:
    print("Hasil: KONDISI NORMAL (CLUSTER A)\n")
elif jB < jC:
    print("Hasil: KONDISI NORMAL (CLUSTER B)\n")
else:
    print("Hasil: KONDISI NORMAL (CLUSTER C)\n")

# UJI COBA
tentukan_cluster(0, 2, 0.5)
tentukan_cluster(-1, 2, -1)
tentukan_cluster("Lanang", 3, 5)

Menguji Titik U: (0.0000000, 2.0000000, 0.5000000)
- Jarak ke Cluster A: 3.3541
- Jarak ke Cluster B: 8.2006
- Jarak ke Cluster C: 3.3541
Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara
CLUSTER A dan CLUSTER C)

Menguji Titik U: (-1.0000000, 2.0000000, -1.0000000)

```



```
- Jarak ke Cluster A: 5.0990
- Jarak ke Cluster B: 9.4340
- Jarak ke Cluster C: 1.7321
Hasil: KONDISI NORMAL (CLUSTER C)
```

Menguji Titik U: (Lanang, 3, 5)

Hasil: KONDISI ERROR (Input harus berupa angka!)

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 5:

```
# KONDISI 2: Kondisi Khusus (Deteksi Cluster yang Jaraknya Sama)
# Ditambahkan pembulatan mikro (round) agar toleransi limit angka pecahan di komputer aman
jA, jB, jC = round(jarak_A, 7), round(jarak_B, 7), round(jarak_C, 7)

if jA == jB and jB == jC:
    print("Hasil: KONDISI KHUSUS (Titik U berada tepat di tengah-tengah antara Cluster A, B, dan C)\n")
elif jA == jB and jA <= jC:
    print("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER B)\n")
elif jA == jC and jA <= jB:
    print("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER C)\n")
elif jB == jC and jB <= jA:
    print("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER B dan CLUSTER C)\n")

# KONDISI 3: Kondisi Normal (Satu Cluster Terdekat)
elif jA < jB and jA < jC:
    print("Hasil: KONDISI NORMAL (CLUSTER A)\n")
elif jB < jC:
    print("Hasil: KONDISI NORMAL (CLUSTER B)\n")
else:
    print("Hasil: KONDISI NORMAL (CLUSTER C)\n")
```

4.6 Program 6

Berikut adalah kode sumber (*source code*) untuk Program 6. Implementasi program ini dibagi ke dalam dua bahasa pemrograman, yaitu R dan Python.

4.6.1 Code R :

```
#PROGRAM NO 6
hitung_interval_konfidensi <- function(p_hat,alpha,n) {
  if (p_hat < 0 || p_hat > 1) {
    cat("Proporsi (p_hat) harus berada di antara 0 dan 1.\n")
  } else if (n <= 0) {
    cat("Ukuran sampel (n) harus lebih besar dari 0.\n")
  } else if (!(alpha == 0.10 || alpha == 0.05)) {
    cat("Nilai alpha harus 0.10 atau 0.05.\n")
  } else {
    # Jika semua valid, baru hitung
    z <- ifelse(alpha == 0.10, 1.645, 1.96)

    margin_of_error <- z * sqrt((p_hat * (1 - p_hat)) / n)
    batas_bawah <- p_hat - margin_of_error
    batas_atas <- p_hat + margin_of_error

    cat("\n--- Hasil Perhitungan ---\n")
    cat(sprintf("\nInterval Konfidensi (%.0f%%): %.4f < p < %.4f\n",
```

```

        (1 - alpha) * 100, batas_bawah, batas_atas))
    }
}

# Menjalankan fungsi
hitung_interval_konfidensi(0.5,0.05,1000)

##
## --- Hasil Perhitungan ---
##
## Interval Konfidensi (95%): 0.4690 < p < 0.5310

hitung_interval_konfidensi(0.5,0.1,50)

##
## --- Hasil Perhitungan ---
##
## Interval Konfidensi (90%): 0.3837 < p < 0.6163

hitung_interval_konfidensi(1.2,0.05,50)

## Proporsi (p_hat) harus berada di antara 0 dan 1.

```

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 6:

```

9  } else {
10     # Jika semua valid, baru hitung
11     z <- ifelse(alpha == 0.10, 1.645, 1.96)
12
13     margin_of_error <- z * sqrt((p_hat * (1 - p_hat)) / n)
14     batas_bawah <- p_hat - margin_of_error
15     batas_atas <- p_hat + margin_of_error
16
17     cat("\n--- Hasil Perhitungan ---\n")
18     cat(sprintf("\nInterval Konfidensi (%.0f%%): %.4f < p < %.4f\n",
19                 (1 - alpha) * 100, batas_bawah, batas_atas))

```

4.6.2 Code Python :

```

import math

def hitung_interval_konfidensi(p_hat, n, alpha):
    try:
        # Konversi input ke tipe data yang sesuai
        p_hat = float(p_hat)
        n = int(n)
        alpha = float(alpha)
        print("--- Program Interval Konfidensi ---")

        # VALIDASI INPUT
        if p_hat < 0 or p_hat > 1:
            print("Error: Proporsi (p_hat) harus berada di antara 0

```

```

dan1.")
    elif n <= 0:
        print("Error: Ukuran sampel (n) harus lebih besar dari 0.")
    elif alpha != 0.10 and alpha != 0.05:
        print("Error: Nilai alpha harus 0.10 atau 0.05.")
    else:
        # Penentuan nilai z
        z = 1.645 if alpha == 0.10 else 1.96

        # Perhitungan Margin of Error
        margin_of_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)

        batas_bawah = p_hat - margin_of_error
        batas_atas = p_hat + margin_of_error

        # Output hasil
        print("\n--- Hasil Perhitungan ---")
        print(f"Interval Konfidensi ({(1 - alpha) * 100:.0f}%): "
              f"{batas_bawah:.4f} < p < {batas_atas:.4f}")

except ValueError:
    # Menangani jika user memasukkan teks (bukan angka)
    print("Error: Input harus berupa angka yang valid.")

# Jalankan program
if __name__ == "__main__":
    hitung_interval_konfidensi(0.6,50,0.05)
    hitung_interval_konfidensi(0.5,50,0.1)
    hitung_interval_konfidensi(1.4,50,0.24)

    --- Program Interval Konfidensi ---

    --- Hasil Perhitungan ---
    Interval Konfidensi (95%): 0.4642 < p < 0.7358
    --- Program Interval Konfidensi ---

    --- Hasil Perhitungan ---
    Interval Konfidensi (90%): 0.3837 < p < 0.6163
    --- Program Interval Konfidensi ---
    Error: Proporsi (p_hat) harus berada di antara 0 dan 1.

```

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan bagian penting dari instruksi Program 6:

```
else:
    # Penentuan nilai z
    z = 1.645 if alpha == 0.10 else 1.96

    # Perhitungan Margin of Error
    margin_of_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)

    batas_bawah = p_hat - margin_of_error
    batas_atas = p_hat + margin_of_error
```

BAB V

PENJELASAN LINE-BY-LINE CODE

5.1 Penjelasan Line-By-Line Code Program 1

Berikut adalah uraian instruksi dari setiap baris kode pada Program 1. Agar pembahasan lebih terstruktur, penjelasan ini dibagi menjadi dua bagian, yaitu implementasi menggunakan bahasa pemrograman R dan Python.

5.1.1 Penjelasan Line-By-Line Code R

Rangkaian kode di bawah ini menunjukkan langkah-langkah penyelesaian Program 1 menggunakan *sintaks* R, beserta penjelasan dari masing-masing baris instruksinya:

No.	Baris Setiap Kode	Penjelasan
1	#Program 1	Komentar untuk menandai bahwa ini adalah kode untuk program 1
2	#Kondisi 1	Komentar untuk menandai bahwa ini adalah kondisi pertama dari program 1
3	teks <- "Ibu sedang masak di dapur, sedangkan ayah mencuci motor. Kucing tidur di atas kasur, sementara burung bernyanyi di luar sangkar."	Memberikan nilai string berupa kalimat pada variabel teks
4	jumlahKalimat <- 0	Memberikan nilai awal pada variabel jumlah_kalimat yaitu nol
5	for(i in 1:nchar(teks)){	Memeriksa teks yang diinput oleh pengguna karakter demi karakter dari awal sampai akhir.
6	if(substr(teks, i, i) == "."){	Mengecek apakah karakter yang saat ini sedang diperiksa adalah tanda titik
7	jumlahKalimat <- jumlahKalimat + 1	Jika kondisi pada baris sebelumnya terpenuhi, maka nilai pada variabel jumlah_kalimat akan ditambah 1.
8	}	Menutup blok kondisi (if)
9	}	Menutup blok perulangan (for)
10	kata <- strsplit(teks, " ")	Memisahkan teks berdasarkan spasi
11	jumlahKata <- length(kata[[1]])	Menghitung banyak elemen sebagai jumlah kata
12	cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata, "kata")	Menampilkan hasil jumlah kalimat dan jumlah kata kepada pengguna.
13	#Kondisi 2	Komentar untuk menandai bahwa ini adalah kondisi kedua dari program 1

14	teks <- "Indonesia merdeka pada tanggal 17 Agustus 1945."	Memberikan nilai string berupa kalimat yang memiliki sebuah angka pada variabel teks
15	jumlahKalimat <- 0	Memberikan nilai awal pada variabel jumlah_kalimat yaitu nol
16	for(i in 1:nchar(teks)){	Memeriksa teks yang diinput oleh pengguna karakter demi karakter dari awal sampai akhir.
17	if(substr(teks, i, i) == "."){	Mengecek apakah karakter yang saat ini sedang diperiksa adalah tanda titik
18	jumlahKalimat <- jumlahKalimat + 1	Jika kondisi pada baris sebelumnya terpenuhi, maka nilai pada variabel jumlah_kalimat akan ditambah 1.
19	}	Menutup blok kondisi (if)
20	}	Menutup blok perulangan (for)
21	kata <- strsplit(teks, " ")	Memisahkan teks berdasarkan spasi
22	jumlahKata <- length(kata[[1]])	Menghitung banyak elemen sebagai jumlah kata
23	cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata, "kata")	Menampilkan hasil jumlah kalimat dan jumlah kata kepada pengguna.
24	#Kondisi 3	Komentar untuk menandai bahwa ini adalah kondisi ketiga dari program 1
25	teks <- "Harga buku ini 50000 namun aku hanya membawa 30000."	Memberikan nilai string berupa kalimat yang memiliki sebuah angka pada variabel teks
26	jumlahKalimat <- 0	Memberikan nilai awal pada variabel jumlah_kalimat yaitu nol
27	for(i in 1:nchar(teks)){	Memeriksa teks yang diinput oleh pengguna karakter demi karakter dari awal sampai akhir.
28	if(substr(teks, i, i) == "."){	Mengecek apakah karakter yang saat ini sedang diperiksa adalah tanda titik
29	jumlahKalimat <- jumlahKalimat + 1	Jika kondisi pada baris sebelumnya terpenuhi, maka nilai pada variabel jumlah_kalimat akan ditambah 1.
30	}	Menutup blok kondisi (if)
31	}	Menutup blok perulangan (for)
32	kata <- strsplit(teks, " ")	Memisahkan teks berdasarkan spasi
33	jumlahKata <- length(kata[[1]])	Menghitung banyak elemen sebagai jumlah kata
34	cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata, "kata")	Menampilkan hasil jumlah kalimat dan jumlah kata kepada pengguna.

5.1.2 Penjelasan Line-By-Line Code Python

Sementara itu, berikut adalah rincian instruksi untuk implementasi Program 1 menggunakan *sintaks* Python:

No.	Baris Setiap Kode	Penjelasan
1	#Program 1	Komentar untuk menandai bahwa ini adalah kode untuk program 1
2	#Kondisi 1	Komentar untuk menandai bahwa ini adalah kondisi pertama dari program 1
3	teks = input("Masukkan Teks:")	Membaca teks dari pengguna dan menyimpannya ke variabel teks
4	#Hitung Kalimat	Komentar untuk menandai perhitungan kalimat
5	jumlah_kalimat = 0	Memberikan nilai awal pada variabel jumlah_kalimat yaitu nol
6	for karakter in teks:	Memeriksa teks yang diinput oleh pengguna karakter demi karakter dari awal sampai akhir.
7	if karakter == "." :	Mengecek apakah karakter yang saat ini sedang diperiksa adalah tanda titik
8	jumlah_kalimat += 1	Jika kondisi pada baris sebelumnya terpenuhi, maka nilai pada variabel jumlah_kalimat akan ditambah 1.
9	#Hitung Kata	Komentar untuk menandai perhitungan kata
10	jumlah_kata = 1	Memberikan nilai awal pada variabel jumlah_kata yaitu satu
11	for karakter in teks :	Memeriksa teks yang diinput oleh pengguna karakter demi karakter dari awal sampai akhir.
12	if karakter == " ":	Mengecek apakah karakter yang saat ini berupa spasi kosong
13	jumlah_kata += 1	Jika kondisi pada baris sebelumnya terpenuhi, maka nilai pada variabel jumlah_kalimat akan ditambah 1.
14	print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.")	Menampilkan hasil jumlah kalimat dan jumlah kata kepada pengguna.
15	#Program 1	Komentar untuk menandai bahwa ini adalah kode untuk program 1
16	#Kondisi 2	Komentar untuk menandai bahwa ini adalah kondisi kedua dari program 1
17	teks = input("Masukkan Teks:")	Membaca teks dari pengguna dan menyimpannya ke variabel teks
18	#Hitung Kalimat	Komentar untuk menandai perhitungan kalimat

19	jumlah_kalimat = 0	Memberikan nilai awal pada variabel jumlah_kalimat yaitu nol
20	for karakter in teks:	Memeriksa teks yang diinput oleh pengguna karakter demi karakter dari awal sampai akhir.
21	if karakter == "." :	Mengecek apakah karakter yang saat ini sedang diperiksa adalah tanda titik
22	jumlah_kalimat += 1	Jika kondisi pada baris sebelumnya terpenuhi, maka nilai pada variabel jumlah_kalimat akan ditambah 1.
23	#Hitung Kata	Komentar untuk menandai perhitungan kata
24	jumlah_kata = 1	Memberikan nilai awal pada variabel jumlah_kata yaitu satu
25	for karakter in teks :	Memeriksa teks yang diinput oleh pengguna karakter demi karakter dari awal sampai akhir.
26	if karakter == " ":	Mengecek apakah karakter yang saat ini berupa spasi kosong
27	jumlah_kata += 1	Jika kondisi pada baris sebelumnya terpenuhi, maka nilai pada variabel jumlah_kalimat akan ditambah 1.
28	print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.")	Menampilkan hasil jumlah kalimat dan jumlah kata kepada pengguna.
29	#Program 1	Komentar untuk menandai bahwa ini adalah kode untuk program 1
30	#Kondisi 3	Komentar untuk menandai bahwa ini adalah kondisi ketiga dari program 1
31	teks = input("Masukkan Teks:")	Membaca teks dari pengguna dan menyimpannya ke variabel teks
32	#Hitung Kalimat	Komentar untuk menandai perhitungan kalimat
33	jumlah_kalimat = 0	Memberikan nilai awal pada variabel jumlah_kalimat yaitu nol
34	for karakter in teks:	Memeriksa teks yang diinput oleh pengguna karakter demi karakter dari awal sampai akhir.
35	if karakter == "." :	Mengecek apakah karakter yang saat ini sedang diperiksa adalah tanda titik
36	jumlah_kalimat += 1	Jika kondisi pada baris sebelumnya terpenuhi, maka nilai pada variabel jumlah_kalimat akan ditambah 1.
37	#Hitung Kata	Komentar untuk menandai perhitungan kata
38	jumlah_kata = 1	Memberikan nilai awal pada variabel jumlah_kata yaitu satu

39	for karakter in teks :	Memeriksa teks yang diinput oleh pengguna karakter demi karakter dari awal sampai akhir.
40	if karakter == " ":	Mengecek apakah karakter yang saat ini berupa spasi kosong
41	jumlah_kata += 1	Jika kondisi pada baris sebelumnya terpenuhi, maka nilai pada variabel jumlah_kalimat akan ditambah 1.
42	print("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.")	Menampilkan hasil jumlah kalimat dan jumlah kata kepada pengguna.

5.2 Penjelasan Line-By-Line Code Program 2

Berikut adalah uraian instruksi dari setiap baris kode pada Program 2. Penjelasan ini dibagi menjadi dua bagian, yaitu untuk bahasa pemrograman R dan Python.

5.2.1 Penjelasan Line-By-Line Code R

Rangkaian kode di bawah ini menjabarkan instruksi per baris untuk Program 2 menggunakan *sintaks* R:

No	Baris Setiap Kode	Penjelasan
1	#Program String_Kalimat	Komentar untuk menandai nama program.
2	K1 <- "Saya tak 'kan menyerah."	Menginisialisasi variabel K1 bertipe <i>string</i> . Teks dibungkus menggunakan tanda kutip ganda ("...") sebagai pembatas luar agar karakter kutip tunggal di dalam kata "'kan" dapat terbaca tanpa menyebabkan <i>error sintaks</i> .
3	K2 <- 'Ia berkata, "Aku menyayangimu."'	Menginisialisasi variabel K2. Teks dibungkus menggunakan tanda kutip tunggal ('...') sebagai pembatas luar karena di dalam teks memuat kutipan kalimat langsung yang menggunakan tanda kutip ganda.
4	K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""	Menginisialisasi variabel K3. Baris ini menggunakan tanda petik ganda sebagai pembungkus teks agar tanda petik tunggal di dalamnya dapat terbaca dan tambahan <i>escape character</i> berupa <i>backslash</i> (\) agar tanda kutip ganda yang mengapit kalimat dibaca oleh program sebagai bagian dari teks, bukan sebagai pembungkusnya.
5	K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."	Menginisialisasi variabel K4 yang memuat kombinasi huruf, angka, dan karakter tanda baca. Teks dibungkus menggunakan tanda

		petik ganda biasa karena tidak ada tanda petik ganda atau tunggal di dalam teks.
6	<code>cat(K1, "\n")</code>	Mencetak isi dari variabel K1 ke layar konsol. Fungsi <code>cat()</code> digunakan agar teks tercetak bersih tanpa tanda kutip dan indeks [1] bawaan R. Argumen <code>\n</code> ditambahkan agar kalimat dicetak pada baris baru yang berbeda.
7	<code>cat(K2, "\n")</code>	Mencetak isi dari variabel K2 ke layar konsol. Fungsi <code>cat()</code> digunakan agar teks tercetak bersih tanpa tanda kutip dan indeks [1] bawaan R. Argumen <code>\n</code> ditambahkan agar kalimat dicetak pada baris baru yang berbeda.
8	<code>cat(K3, "\n")</code>	Mencetak isi dari variabel K3 ke layar konsol. Fungsi <code>cat()</code> digunakan agar teks tercetak bersih tanpa tanda kutip dan indeks [1] bawaan R. Argumen <code>\n</code> ditambahkan agar kalimat dicetak pada baris baru yang berbeda.
9	<code>cat(K4, "\n")</code>	Mencetak isi dari variabel K4 ke layar konsol. Fungsi <code>cat()</code> digunakan agar teks tercetak bersih tanpa tanda kutip dan indeks [1] bawaan R. Argumen <code>\n</code> ditambahkan agar kalimat dicetak pada baris baru yang berbeda.

5.2.2 Penjelasan Line-By-Line Code Python

Berikut adalah penjelasan instruksi per baris untuk Program 2 versi Python:

No	Baris Setiap Kode	Penjelasan
1	<code>#Program String_Kalimat</code>	Komentar untuk menandai nama program.
2	<code>K1 = "Saya tak 'kan menyerah."</code>	Menginisialisasi variabel K1 bertipe <i>string</i> . Teks dibungkus menggunakan tanda kutip ganda (“...”) sebagai pembatas luar agar karakter kutip tunggal di dalam kata “’kan” dapat terbaca tanpa menyebabkan <i>error</i> sintaks.
3	<code>K2 = 'Ia berkata, "Aku menyayangimu."'</code>	Menginisialisasi variabel K2. Teks dibungkus menggunakan tanda kutip tunggal (‘...’) sebagai pembatas luar karena di dalam teks memuat kutipan kalimat langsung yang menggunakan tanda kutip ganda.
4	<code>K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\\""</code>	Menginisialisasi variabel K3. Baris ini menggunakan tanda petik ganda sebagai pembungkus teks agar tanda petik tunggal di dalamnya dapat terbaca dan tambahan <i>escape character</i> berupa backslash (\) agar tanda

		kutip ganda yang mengapit kalimat dibaca oleh program sebagai bagian dari teks, bukan sebagai pembungkusnya.
5	K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."	Menginisialisasi variabel K4 yang memuat kombinasi huruf, angka, dan karakter tanda baca. Teks dibungkus menggunakan tanda petik ganda biasa karena tidak ada tanda petik ganda atau tunggal di dalam teks.
6	print(K1)	Mencetak isi dari variabel K1 ke layar konsol. Fungsi print() digunakan untuk menampilkan teks ke layar dan secara otomatis tercetak pada baris baru, sehingga tidak memerlukan tambahan argumen \n.
7	print(K1)	Mencetak isi dari variabel K2 ke layar konsol. Fungsi print() digunakan untuk menampilkan teks ke layar dan secara otomatis tercetak pada baris baru, sehingga tidak memerlukan tambahan argumen \n.
8	print(K1)	Mencetak isi dari variabel K3 ke layar konsol. Fungsi print() digunakan untuk menampilkan teks ke layar dan secara otomatis tercetak pada baris baru, sehingga tidak memerlukan tambahan argumen \n.
9	print(K1)	Mencetak isi dari variabel K4 ke layar konsol. Fungsi print() digunakan untuk menampilkan teks ke layar dan secara otomatis tercetak pada baris baru, sehingga tidak memerlukan tambahan argumen \n.

5.3 Penjelasan Line-By-Line Code Program 3

Berikut adalah uraian instruksi dari setiap baris kode pada Program 3. Penjelasan ini dibagi menjadi dua bagian, yaitu untuk bahasa pemrograman R dan Python.

5.3.1 Penjelasan Line-By-Line Code R

Berikut adalah rincian instruksi yang digunakan untuk menjalankan Program 3 dengan *sintaks* R:

No.	Baris Setiap Kode	Penjelasan
1	#Program 3	Komentar untuk menandai bahwa ini adalah kode untuk program 3
2	#Kondisi 1	Komentar untuk menandai bahwa ini adalah kondisi pertama dari program 3
3	a <- 1	Memberikan nilai 1 pada variabel a

4	<code>b <- 2</code>	Memberikan nilai 2 pada variabel b
5	<code>c <- 1</code>	Memberikan nilai 1 pada variabel c
6	<code>D <- b^2 - 4*a*c</code>	Menghitung nilai D (Diskriminan)
7	<code>if (D == 0) {</code>	Memeriksa apakah nilai D sama dengan 0
8	<code>x <- -b / (2*a)</code>	Menghitung akar kembar
9	<code>cat("Nilai x kembar =", round(x, 3), "\n")</code>	Menampilkan string dan nilai akar kembar yang dibulatkan 3 angka di belakang koma
10	<code>} else if (D > 0) {</code>	Kondisi alternatif apabila nilai D lebih besar dari 0
11	<code>x1 <- (-b + sqrt(D)) / (2*a)</code>	Menghitung akar pertama
12	<code>x2 <- (-b - sqrt(D)) / (2*a)</code>	Menghitung akar kedua
13	<code>cat("Akar real:\n")</code>	Menampilkan string untuk judul
14	<code>cat("x1 =", round(x1, 3), "\n")</code>	Menampilkan nilai akar pertama yang dibulatkan 3 angka di belakang koma
15	<code>cat("x2 =", round(x2, 3), "\n")</code>	Menampilkan nilai akar kedua yang dibulatkan 3 angka di belakang koma
16	<code>} else {</code>	Kondisi alternatif apabila kedua kondisi di atas tidak terpenuhi
17	<code>cat("Persamaan hanya memiliki akar imajiner\n")</code>	Menampilkan pesan bahwa "Persamaan hanya memiliki akar imajiner"
18	<code>}</code>	Menutup kondisi percabangan
19	<code>#Kondisi 2</code>	Komentar untuk menandai bahwa ini adalah kondisi kedua dari program 3
20	<code>a <- 1</code>	Memberikan nilai 1 pada variabel a
21	<code>b <- -5</code>	Memberikan nilai -5 pada variabel b
22	<code>c <- 6</code>	Memberikan nilai 6 pada variabel c
23	<code>D <- b^2 - 4*a*c</code>	Menghitung nilai D (Diskriminan)
24	<code>if (D == 0) {</code>	Memeriksa apakah nilai D sama dengan 0
25	<code>x <- -b / (2*a)</code>	Menghitung akar kembar
26	<code>cat("Nilai x kembar =", round(x, 3), "\n")</code>	Menampilkan string dan nilai akar kembar yang dibulatkan 3 angka di belakang koma
27	<code>} else if (D > 0) {</code>	Kondisi alternatif apabila nilai D lebih besar dari 0
28	<code>x1 <- (-b + sqrt(D)) / (2*a)</code>	Menghitung akar pertama
29	<code>x2 <- (-b - sqrt(D)) / (2*a)</code>	Menghitung akar kedua
30	<code>cat("Akar real:\n")</code>	Menampilkan string untuk judul
31	<code>cat("x1 =", round(x1, 3), "\n")</code>	Menampilkan nilai akar pertama yang dibulatkan 3 angka di belakang koma
32	<code>cat("x2 =", round(x2, 3), "\n")</code>	Menampilkan nilai akar kedua yang dibulatkan 3 angka di belakang koma

33	} else {	Kondisi alternatif apabila kedua kondisi di atas tidak terpenuhi
34	cat("Persamaan hanya memiliki akar imajiner\n")	Menampilkan pesan bahwa "Persamaan hanya memiliki akar imajiner"
35	}	Menutup kondisi percabangan
36	#Kondisi 3	Komentar untuk menandai bahwa ini adalah kondisi ketiga dari program 3
37	a <- 1	Memberikan nilai 1 pada variabel a
38	b <- 2	Memberikan nilai 2 pada variabel b
39	c <- 5	Memberikan nilai 5 pada variabel c
40	D <- b^2 - 4*a*c	Menghitung nilai D (Diskriminan)
41	if (D == 0) {	Memeriksa apakah nilai D sama dengan 0
42	x <- -b / (2*a)	Menghitung akar kembar
43	cat("Nilai x kembar =", round(x, 3), "\n")	Menampilkan string dan nilai akar kembar yang dibulatkan 3 angka di belakang koma
44	} else if (D > 0) {	Kondisi alternatif apabila nilai D lebih besar dari 0
45	x1 <- (-b + sqrt(D)) / (2*a)	Menghitung akar pertama
46	x2 <- (-b - sqrt(D)) / (2*a)	Menghitung akar kedua
47	cat("Akar real:\n")	Menampilkan string untuk judul
48	cat("x1 =", round(x1, 3), "\n")	Menampilkan nilai akar pertama yang dibulatkan 3 angka di belakang koma
49	cat("x2 =", round(x2, 3), "\n")	Menampilkan nilai akar kedua yang dibulatkan 3 angka di belakang koma
50	} else {	Kondisi alternatif apabila kedua kondisi di atas tidak terpenuhi
51	cat("Persamaan hanya memiliki akar imajiner\n")	Menampilkan pesan bahwa "Persamaan hanya memiliki akar imajiner"
52	}	Menutup kondisi percabangan

5.3.2 Penjelasan Line-By-Line Code Python

Adapun untuk versi Python, penjabaran instruksinya dapat dilihat di bawah ini:

No.	Baris Setiap Kode	Penjelasan
1	# Program 3	Komentar untuk menandai bahwa ini adalah kode untuk program 3
2	# Menghitung akar-akar persamaan kuadrat	Komentar untuk menandai bahwa ini adalah kode untuk menghitung akar-akar persamaan kuadrat
3	a = float(input("Masukkan nilai a: "))	Membaca angka dengan tipe data float dari pengguna dan menyimpannya ke variabel a.

4	<code>b = float(input("Masukkan nilai b: "))</code>	Membaca angka dengan tipe data float dari pengguna dan menyimpannya ke variabel b.
5	<code>c = float(input("Masukkan nilai c: "))</code>	Membaca angka dengan tipe data float dari pengguna dan menyimpannya ke variabel c.
6	<code>D = b**2 - 4*a*c</code>	Menghitung nilai D (Diskriminan)
7	<code>if D == 0:</code>	Memeriksa apakah nilai D sama dengan 0
8	<code> x = -b / (2 * a)</code>	Menghitung akar kembar
9	<code> print("Nilai x kembar =", round(x, 3))</code>	Menampilkan string dan nilai akar kembar yang dibulatkan 3 angka di belakang koma
10	<code>elif D > 0:</code>	Kondisi alternatif apabila nilai D lebih besar dari 0
11	<code> x1 = (-b + D**0.5) / (2 * a)</code>	Menghitung akar pertama
12	<code> x2 = (-b - D**0.5) / (2 * a)</code>	Menghitung akar kedua
13	<code> print("Akar real:")</code>	Menampilkan string untuk judul
14	<code> print("x1 =", round(x1, 3))</code>	Menampilkan nilai akar pertama yang dibulatkan 3 angka di belakang koma
15	<code> print("x2 =", round(x2, 3))</code>	Menampilkan nilai akar kedua yang dibulatkan 3 angka di belakang koma
16	<code>else:</code>	Kondisi alternatif apabila kedua kondisi di atas tidak terpenuhi
17	<code> print("Persamaan hanya memiliki akar imajiner")</code>	Menampilkan pesan bahwa "Persamaan hanya memiliki akar imajiner"

5.4 Penjelasan Line-By-Line Code Program 4

Bagian ini menjelaskan instruksi per baris pada Program 4. Penjelasan dibagi ke dalam dua bahasa pemrograman, yaitu R dan Python.

5.4.1 Penjelasan Line-By-Line Code R

Berikut adalah penjelasan instruksi Program 4 menggunakan *sintaks* R:

No.	Baris Setiap Kode	Penjelasan
1	<code>#Program NIP_ASN</code>	Komentar untuk menandai nama program.
2	<code>#Inisialisasi kamus bulan dan batas hari</code>	Komentar untuk menjelaskan kode di bawahnya merupakan proses inisialisasi data bulan dan batas hari.
3	<code>data_bulan <- c("Januari", "Februari", "Maret", "April", "Mei", "Juni", "Juli", "Agustus", "September", "Oktober", "November", "Desember")</code>	Menginisialisasi variabel data_bulan bertipe vektor untuk menyimpan nama-nama bulan sebagai acuan dalam menafsirkan angka bulan dari NIP.

4	<code>data_hari <- c(31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)</code>	Menginisialisasi variabel <code>data_hari</code> bertipe vektor numerik untuk menyimpan batas jumlah hari maksimal setiap bulannya (Februari diatur <i>default</i> 28 hari)
5	<code>while (TRUE) {</code>	Memulai perulangan (<i>loop</i>) tanpa batas agar program terus meminta input selama pengguna memasukkan data yang tidak valid.
6	<code>#Menerima input dari user</code>	Komentar untuk menjelaskan kode di bawahnya berfungsi meminta data NIP dari pengguna.
7	<code>NIP <- readline(prompt = "Masukkan 18 digit NIP Anda: ")</code>	Membaca input teks NIP dari pengguna melalui konsol dan menyimpannya ke dalam variabel <code>NIP</code> .
8	<code>tahun <- as.numeric(substr(NIP, 1, 4)) #Indeks R mulai dari 1</code>	Mengekstrak 4 karakter pertama NIP untuk data tahun lahir, mengonversinya ke tipe data numerik, dan disimpan ke variabel <code>tahun</code> .
9	<code>bulan <- as.numeric(substr(NIP, 5, 6))</code>	Mengekstrak karakter ke-5 hingga ke-6 dari NIP untuk data bulan lahir, mengonversinya ke tipe data numerik, dan disimpan ke variabel <code>bulan</code> .
10	<code>tanggal <- as.numeric(substr(NIP, 7, 8))</code>	Mengekstrak karakter ke-7 hingga ke-8 dari NIP untuk data tanggal lahir, mengonversinya ke tipe data numerik, dan disimpan ke variabel <code>tanggal</code> .
11	<code>#Validasi rentang bulan</code>	Komentar untuk menjelaskan kode di bawahnya berfungsi memvalidasi batas rentang bulan.
12	<code>if (bulan < 1 bulan > 12){</code>	Kondisi jika bulan kurang dari 1 atau lebih dari 12.
13	<code>cat("NIP tidak valid, harap masukkan ulang.\n\n")</code>	Mencetak teks peringatan NIP tidak valid dan meminta untuk memasukkan ulang, karena input bulan lahir dinilai tidak rasional atau kondisi di atasnya terpenuhi.
14	<code>next</code>	Perintah untuk menghentikan sisa proses dan langsung mengarahkan program kembali ke awal perulangan <code>while</code> untuk meminta NIP baru.
15	<code>}</code>	Tanda kurung kurawal penutup untuk mengakhiri blok kondisi validasi rentang bulan.
16	<code>#Mengambil data dari kamus</code>	Komentar untuk menjelaskan kode di bawahnya proses penentuan nama bulan dan

		penentuan batas hari maksimal berdasarkan hasil ekstraksi.
17	<code>nama_bulan <- data_bulan[bulan]</code>	Mencocokkan angka bulan dengan data_bulan untuk mendapatkan nama bulan dalam bentuk teks.
18	<code>max_hari <- data_hari[bulan]</code>	Mengambil nilai batas maksimal hari dari data_hari sesuai dengan indeks pada angka bulan.
19	<code>#Cek tahun kabisat (update batas Februari)</code>	Komentar untuk menjelaskan kode di bawahnya berfungsi mengecek tahun kabisat dan perubahan batas pada bulan Februari.
20	<code>if (bulan == 2 & (tahun %% 400 == 0 (tahun %% 4 == 0 & tahun %% 100 != 0))) {</code>	Kondisi jika bulan Februari dan tahunnya adalah tahun kabisat setelah dilakukan perhitungan matematis.
21	<code>max_hari = 29</code>	Mengubah nilai batas hari maksimal khusus untuk bulan Februari menjadi 29 hari jika kondisi tahun kabisat terpenuhi.
22	<code>}</code>	Tanda kurung kurawal penutup untuk mengakhiri blok pengecekan tahun kabisat.
23	<code>#Validasi batas tanggal</code>	Komentar untuk menjelaskan kode di bawahnya berfungsi untuk validasi batas tanggal lahir.
24	<code>if (tanggal < 1 tanggal > max_hari) {</code>	Kondisi jika tanggal kurang dari 1 atau tanggal melebihi maksimal hari untuk bulan tersebut.
25	<code>cat("NIP tidak valid, harap masukkan ulang.\n\n")</code>	Mencetak teks peringatan NIP tidak valid dan meminta untuk memasukkan ulang, karena input tanggal lahir dinilai tidak rasional atau kondisi di atasnya terpenuhi.
26	<code>next</code>	Perintah untuk menghentikan sisa proses dan langsung mengarahkan program kembali ke awal perulangan while untuk meminta NIP baru.
27	<code>}</code>	Tanda kurung kurawal penutup untuk mengakhiri blok kondisi validasi batas tanggal.
28	<code>#Jika semua valid</code>	Komentar untuk menjelaskan kode di bawahnya akan berjalan ketika data input dinilai valid dalam proses validasi sebelumnya.
29	<code>cat("Tanggal lahir Anda:", tanggal, nama_bulan, tahun, "\n")</code>	Menampilkan hasil ekstraksi, yaitu informasi tanggal lahir ASN ke layar secara lengkap

		yang mencakup tanggal, nama bulan, dan tahun lahir.
30	<code>break</code>	Perintah untuk menghentikan paksa dan keluar dari blok perulangan <code>while</code> karena data yang dinilai valid sudah didapatkan.
31	<code>}</code>	Tanda kurung kurawal penutup untuk mengakhiri keseluruhan blok perulangan utama <code>while</code> .

5.4.2 Penjelasan Line-By-Line Code Python

Berikut adalah penjelasan instruksi Program 4 menggunakan *sintaks* Python:

No.	Baris Setiap Kode	Penjelasan
1	<code>#Program NIP_ASN</code>	Komentar untuk menandai nama program.
2	<code>#Inisialisasi kamus bulan dan batas hari</code>	Komentar untuk menjelaskan kode di bawahnya merupakan proses inisialisasi data bulan dan batas hari.
3	<code>data_bulan = { 1:"Januari", 2:"Februari", 3:"Maret", 4:"April", 5:"Mei", 6:"Juni", 7:"Juli", 8:"Agustus", 9:"September", 10:"Oktober", 11:"November", 12:"Desember" }</code>	Menginisialisasi variabel <code>data_bulan</code> bertipe <i>dictionary</i> untuk menyimpan nama-nama bulan sebagai acuan dalam menafsirkan angka bulan dari NIP.
4	<code>data_hari = { 1:31, 2:28, 3:31, 4:30, 5:31, 6:30, 7:31, 8:31, 9:30, 10:31, 11:30, 12:31 }</code>	Menginisialisasi variabel <code>data_hari</code> bertipe <i>dictionary</i> untuk menyimpan batas jumlah hari maksimal setiap bulannya (Februari diatur <i>default</i> 28 hari)
5	<code>while True:</code>	Memulai perulangan (<i>loop</i>) tanpa batas agar program terus meminta input selama pengguna memasukkan data yang tidak valid.
6	<code>NIP = input("Masukkan 18 digit NIP Anda:")</code>	Membaca input teks NIP dari pengguna melalui konsol dan menyimpannya ke dalam variabel <code>NIP</code> .
7	<code>tahun = int(NIP[0:4]) #Indeks Python mulai dari 0</code>	Mengekstrak karakter dari indeks 0 hingga sebelum 4 dari NIP untuk data tahun lahir, mengonversinya ke tipe data numerik, dan disimpan ke variabel <code>tahun</code> .
8	<code>bulan = int(NIP[4:6])</code>	Mengekstrak karakter dari indeks 4 hingga sebelum 6 dari NIP untuk data bulan lahir,

		mengonversinya ke tipe data numerik, dan disimpan ke variabel bulan.
9	<code>tanggal = int(NIP[6:8])</code>	Mengekstrak karakter dari indeks 6 hingga sebelum 8 dari NIP untuk data tanggal lahir, mengonversinya ke tipe data numerik, dan disimpan ke variabel tanggal.
10	<code>#Validasi rentang bulan</code>	Komentar untuk menjelaskan kode di bawahnya berfungsi memvalidasi batas rentang bulan.
11	<code>if bulan < 1 or bulan > 12:</code>	Kondisi jika bulan kurang dari 1 atau lebih dari 12.
12	<code>print("NIP tidak valid, harap masukkan ulang.")</code>	Mencetak teks peringatan NIP tidak valid dan meminta untuk memasukkan ulang, karena input bulan lahir dinilai tidak rasional atau kondisi di atasnya terpenuhi.
13	<code>continue</code>	Perintah untuk menghentikan sisa proses dan langsung mengarahkan program kembali ke awal perulangan while untuk meminta NIP baru.
14	<code>#Mengambil data dari kamus</code>	Komentar untuk menjelaskan kode di bawahnya proses penentuan nama bulan dan pencarian batas hari maksimal berdasarkan hasil ekstraksi.
15	<code>nama_bulan = data_bulan[bulan]</code>	Mencocokkan angka bulan dengan data_bulan untuk mendapatkan nama bulan dalam bentuk teks.
16	<code>max_hari = data_hari[bulan]</code>	Mengambil nilai batas maksimal hari dari data_hari sesuai dengan indeks pada angka bulan.
17	<code>#Cek tahun kabisat (update batas Februari)</code>	Komentar untuk menjelaskan kode di bawahnya berfungsi mengecek tahun kabisat dan perubahan batas pada bulan Februari.
18	<code>if bulan == 2 and (tahun % 400 == 0 or (tahun % 4 == 0 and tahun % 100 != 0)):</code>	Kondisi jika bulan Februari dan tahunnya adalah tahun kabisat setelah dilakukan perhitungan matematis.
19	<code>max_hari = 29</code>	Mengubah nilai batas hari maksimal khusus untuk bulan Februari menjadi 29 hari jika kondisi tahun kabisat terpenuhi.
20	<code>#Validasi batas tanggal</code>	Komentar untuk menjelaskan kode di bawahnya berfungsi untuk validasi batas tanggal lahir.

21	if tanggal <1 or tanggal > max_hari:	Kondisi jika tanggal kurang dari 1 atau tanggal melebihi maksimal hari untuk bulan tersebut.
22	print("NIP tidak valid, harap masukkan ulang.")	Mencetak teks peringatan NIP tidak valid dan meminta untuk memasukkan ulang, karena input tanggal lahir dinilai tidak rasional atau kondisi di atasnya terpenuhi.
23	continue	Perintah untuk menghentikan sisa proses dan langsung mengarahkan program kembali ke awal perulangan while untuk meminta NIP baru.
24	#Jika semua valid	Komentar untuk menjelaskan kode di bawahnya akan berjalan ketika data input dinilai valid dalam proses validasi sebelumnya.
25	print("Tanggal lahir Anda:", tanggal, nama_bulan, tahun)	Menampilkan hasil ekstraksi, yaitu informasi tanggal lahir ASN ke layar secara lengkap yang mencakup tanggal, nama bulan, dan tahun lahir.
26	break	Perintah untuk menghentikan paksa dan keluar dari blok perulangan while karena data yang dinilai valid sudah didapatkan.

5.5 Penjelasan Line-By-Line Code Program 5

Berikut adalah uraian instruksi per baris untuk Program 5. Penjelasan mencakup implementasi dalam bahasa pemrograman R dan Python.

5.5.1 Penjelasan Line-By-Line Code R

Rincian instruksi Program 5 menggunakan *sintaks* R adalah sebagai berikut:

No.	Baris Setiap Kode	Penjelasan
1	tentukan_cluster <- function(x1, x2, x3) {	Mendefinisikan sebuah fungsi bernama tentukan_cluster di R menggunakan operator <i>assignment</i> (<-) dengan tiga buah argumen input.
2	# KONDISI 1: Validasi Error Input (Bukan Angka)	Komentar Dokumentasi: Menandai blok kode proteksi input data dari kesalahan tipe karakter non-numerik pada bahasa pemrograman R.
3	if (!is.numeric(x1) !is.numeric(x2) !is.numeric(x3)) {	Melakukan validasi tipe data menggunakan fungsi bawaan is.numeric() dan operator OR (). Jika salah satu input bukan angka, program masuk ke penanganan galat.

4	<code>cat(sprintf("Menguji Titik U: (%s, %s, %s)\n", as.character(x1), as.character(x2), as.character(x3)))</code>	Mengonversi objek input menjadi karakter teks menggunakan <code>as.character()</code> lalu mencetaknya via perpaduan fungsi pemformatan string <code>sprintf()</code> dan perintah <code>cat()</code> .
5	<code>cat("Hasil: KONDISI ERROR (Input harus berupa angka!)\n\n")</code>	Menampilkan teks pemberitahuan error ke layar konsol RStudio jika syarat validasi numerik dilanggar.
6	<code>}else{</code>	Membuka blok alternatif <code>else</code> yang akan dijalankan apabila semua data masukan terverifikasi lolos sebagai tipe data angka.
7	<code># Tetapkan koordinat pusat cluster tetap</code>	Komentar dokumentasi yang menjelaskan deklarasi titik pusat cluster konstan A, B, dan C sesuai ketentuan soal ke dalam memori R.
8	<code>A <- c(2, 1, 3)</code>	Membuat objek pusat cluster A menggunakan fungsi penggabung <code>c()</code> untuk membentuk struktur data berupa kumpulan angka (vector).
9	<code>B <- c(1, -4, 6)</code>	Membuat objek pusat cluster B menggunakan fungsi penggabung <code>c()</code> untuk membentuk struktur data berupa kumpulan angka (vector).
10	<code>C <- c(-2, 3, -2)</code>	Membuat objek pusat cluster C menggunakan fungsi penggabung <code>c()</code> untuk membentuk struktur data berupa kumpulan angka (vector).
11	<code># Gabungkan parameter menjadi vektor U</code>	Komentar dokumentasi sebagai penanda proses penyatuan parameter terpisah menjadi satu kesatuan objek spasial 3D.
12	<code>U <- c(x1, x2, x3)</code>	Menggabungkan komponen koordinat input pengguna <code>x1</code> , <code>x2</code> , <code>x3</code> ke dalam satu kesatuan objek <i>vector</i> 3 dimensi bernama U.
13	<code># Fungsi internal untuk menghitung jarak Euclidean</code>	Komentar dokumentasi yang menjelaskan pembuatan fungsi lokal khusus lingkungan internal di R untuk keperluan teknik modularisasi program.
14	<code>hitung_jarak <- function(titik1, titik2) {</code>	Mendefinisikan bodi fungsi internal <code>hitung_jarak</code> yang menerima parameter dua objek vektor.
15	<code>return(sqrt(sum((titik1 - titik2)^2)))</code>	Memanfaatkan fitur operasi <i>vectorized</i> di R; mengurangi kedua vektor, menguadratkannya, menjumlahkan total elemen dengan <code>sum()</code> , lalu melakukan akar kuadrat <code>sqrt()</code> .
16	<code>}</code>	Menutup fungsi internal.
17	<code># Hitung jarak dari U ke setiap cluster</code>	Komentar dokumentasi yang menandai alur eksekusi fungsi <code>hitung_jarak</code> spasial terhadap target cluster A, B, dan C.

18	<code>jarak_A <- hitung_jarak(U, A)</code>	Memanggil fungsi internal <code>hitung_jarak</code> untuk mendapatkan hasil jarak mutlak antara vektor titik U dengan pusat cluster A.
19	<code>jarak_B <- hitung_jarak(U, B)</code>	Memanggil fungsi internal <code>hitung_jarak</code> untuk mendapatkan hasil jarak mutlak antara vektor titik U dengan pusat cluster B.
20	<code>jarak_C <- hitung_jarak(U, C)</code>	Memanggil fungsi internal <code>hitung_jarak</code> untuk mendapatkan hasil jarak mutlak antara vektor titik U dengan pusat cluster C.
21	<code># Cetak informasi titik dan jarak</code>	Komentar dokumentasi sebagai penanda bagian pemformatan dan visualisasi output teks ringkasan hasil perhitungan jarak di konsol R.
22	<code>cat(sprintf("Menguji Titik U: (%.1f, %.1f, %.1f)\n", x1, x2, x3))</code>	Menampilkan teks pelaporan koordinat U yang sedang diuji dengan format visualisasi satu angka desimal (<code>%.1f</code>).
23	<code>cat(sprintf("- Jarak ke Cluster A: %.4f\n", jarak_A))</code>	Mencetak ringkasan informasi hasil hitung jarak ke cluster A pada konsol dengan format tampilan 4 angka pecahan desimal di belakang koma (<code>%.4f</code>).
24	<code>cat(sprintf("- Jarak ke Cluster B: %.4f\n", jarak_B))</code>	Mencetak ringkasan informasi hasil hitung jarak ke cluster B pada konsol dengan format tampilan 4 angka pecahan desimal di belakang koma (<code>%.4f</code>).
25	<code>cat(sprintf("- Jarak ke Cluster C: %.4f\n", jarak_C))</code>	Mencetak ringkasan informasi hasil hitung jarak ke cluster C pada konsol dengan format tampilan 4 angka pecahan desimal di belakang koma (<code>%.4f</code>).
26	<code># KONDISI 2: Kondisi Khusus (Deteksi Cluster yang Jaraknya Sama)</code>	Komentar dokumentasi yang menandai transisi program ke dalam tahap pengujian logika kasus seimbang atau kembar (<i>tie-breaker</i>).
27	<code>if (jarak_A == jarak_B && jarak_B == jarak_C) {</code>	Memasuki pengujian kondisi khusus menggunakan operator logika AND (<code>&&</code>) untuk mengecek apakah ada hasil jarak ke cluster yang bernilai kembar atau sama besar pada ketiga cluster tersebut.
28	<code>cat("Hasil: KONDISI KHUSUS (Titik U berada tepat di tengah-tengah antara Cluster A, B, dan C)\n\n")</code>	Mencetak Hasil: KONDISI KHUSUS (Titik U berada tepat di tengah-tengah antara Cluster A, B, dan C).
29	<code>} else if (jarak_A == jarak_B && jarak_A <= jarak_C) {</code>	Mengevaluasi variasi kondisi khusus lainnya di mana titik U berada di posisi seimbang di antara dua cluster terdekat yang bernilai kembar, yaitu cluster A dan B.

30	cat("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER B)\n\n")	Mencetak Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER B).
31	} else if (jarak_A == jarak_C && jarak_A <= jarak_B) {	Mengevaluasi variasi kondisi khusus lainnya di mana titik U berada di posisi seimbang di antara dua cluster terdekat yang bernilai kembar, yaitu cluster A dan C.
32	cat("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER C)\n\n")	Mencetak Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER C).
33	} else if (jarak_B == jarak_C && jarak_B <= jarak_A) {	Mengevaluasi variasi kondisi khusus lainnya di mana titik U berada di posisi seimbang di antara dua cluster terdekat yang bernilai kembar, yaitu cluster B dan C.
34	cat("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER B dan CLUSTER C)\n\n")	Mencetak Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER B dan CLUSTER C).
35	}	Kurung kurawal penutup untuk mengakhiri ruang lingkup cakupan blok else if kondisi khusus terakhir.
36	# KONDISI 3: Kondisi Normal (Satu Cluster Terdekat)	Komentar dokumentasi yang menandai blok keputusan akhir untuk kondisi normal (nilai jarak minimum mutlak tunggal).
37	else if (jarak_A < jarak_B && jarak_A < jarak_C) {	Mengevaluasi jika U tidak termasuk kondisi khusus dan input tervalidasi int atau float, selanjutnya nilai jarak A lebih kecil dari nilai jarak B dan nilai jarak C lakukan perintah pada baris setelah ini.
38	cat("Hasil: KONDISI NORMAL (CLUSTER A)\n\n")	Mencetak Hasil: KONDISI NORMAL (CLUSTER A).
39	} else if (jarak_B < jarak_C) {	Mengevaluasi jika nilai jarak A tidak paling kecil dan nilai jarak B lebih kecil dari nilai jarak C lakukan perintah pada baris setelah ini.
40	cat("Hasil: KONDISI NORMAL (CLUSTER B)\n\n")	Mencetak Hasil: KONDISI NORMAL (CLUSTER B).
41	} else {	Mengevaluasi jika nilai jarak C yang paling kecil.
42	cat("Hasil: KONDISI NORMAL (CLUSTER C)\n\n")	Mencetak Hasil: KONDISI NORMAL (CLUSTER C).

43	}	Kurung kurawal penutup untuk mengakhiri ruang lingkup cakupan blok else kondisi normal terakhir.
44	}	Kurung kurawal penutup untuk mengakhiri ruang lingkup cakupan blok else utama yang berjalan apabila semua data masukan terverifikasi lolos sebagai tipe data angka.
45	}	Kurung kurawal penutup untuk menutup logika fungsi tentukan_cluster.
46	# Uji COBA KONDISI	Komentar dokumentasi pengujian fungsi dengan berbagai kasus data.
47	tentukan_cluster(0, 2, 0.5)	Memanggil fungsi untuk kondisi khusus dengan U (0, 2, 0.5).
48	tentukan_cluster(-1, 2, -1)	Memanggil fungsi untuk kondisi normal dengan U (-1, 2, -1).
49	tentukan_cluster("Lanang",3,5)	Memanggil fungsi untuk kondisi input error dengan U ("Lanang", 3, 5).

5.5.2 Penjelasan Line-By-Line Code Python

Sementara itu, rincian instruksi Program 5 menggunakan *sintaks* Python adalah sebagai berikut:

No.	Baris Setiap Kode	Penjelasan
1	import math	Mengimpor pustaka (<i>library</i>) bawaan Python bernama math untuk menyediakan fungsi matematika tingkat lanjut, seperti fungsi sqrt() (akar kuadrat).
2	def tentukan_cluster(x1, x2, x3):	Mendefinisikan fungsi utama bernama tentukan_cluster yang menerima 3 parameter input (x1, x2, x3) sebagai representasi koordinat titik U di ruang 3D.
3	# KONDISI 1: Validasi Error Input (Bukan Angka)	Komentar program untuk memperjelas bahwa blok di bawahnya berfungsi sebagai filter keamanan input data.
4	# Memeriksa apakah ada parameter yang bukan int atau float	Komentar program untuk menginformasikan logika pemeriksaan apakah ada parameter yang bukan int atau float.
5	if not (isinstance(x1, (int, float)) and	Memeriksa tipe data parameter satu menggunakan fungsi isinstance(). Jika parameter satu bukan bilangan bulat (int) atau desimal (float), kondisi bernilai True.
6	isinstance(x2, (int, float)) and	Memeriksa tipe data parameter dua menggunakan fungsi isinstance(). Jika

		parameter dua bukan bilangan bulat (int) atau desimal (float), kondisi bernilai True.
7	<code>isinstance(x3, (int, float)):</code>	Memeriksa tipe data parameter tiga menggunakan fungsi <code>isinstance()</code> . Jika parameter tiga bukan bilangan bulat (int) atau desimal (float), kondisi bernilai True.
8	<code>print(f"Menguji Titik U: ({x1}, {x2}, {x3})")</code>	Mencetak string format (<i>f-string</i>) ke layar untuk menunjukkan data koordinat mentah yang diinput oleh pengguna.
9	<code>print("Hasil: KONDISI ERROR (Input harus berupa angka!)\n")</code>	Mencetak pesan peringatan atau informasi kesalahan (<i>error message</i>) bahwa data yang dimasukkan tidak valid.
10	<code>return # Keluar dari fungsi dengan aman</code>	Menghentikan paksa eksekusi fungsi secara prematur agar program langsung keluar dan tidak melanjutkan proses perhitungan matematika yang bisa memicu <i>crash</i> serta terdapat komentar program yang menginformasikan fungsi <i>return</i> agar dapat keluar dari fungsi dengan aman.
11	<code># Tetapkan koordinat pusat cluster tetap</code>	Komentar program untuk menginformasikan penginisialisasian koordinat pusat tetap setiap clusternya.
12	<code>A = (2, 1, 3)</code>	Menginisialisasi koordinat pusat cluster statis A sesuai ketentuan soal ke dalam struktur data <i>tuple</i> (bersifat konstan/tidak bisa diubah).
13	<code>B = (1, -4, 6)</code>	Menginisialisasi koordinat pusat cluster statis B sesuai ketentuan soal ke dalam struktur data <i>tuple</i> (bersifat konstan/tidak bisa diubah).
14	<code>C = (-2, 3, -2)</code>	Menginisialisasi koordinat pusat cluster statis C sesuai ketentuan soal ke dalam struktur data <i>tuple</i> (bersifat konstan/tidak bisa diubah).
15	<code># Fungsi internal untuk menghitung jarak Euclidean</code>	Komentar dokumentasi untuk menandai implementasi prinsip modularisasi, yaitu pembuatan fungsi di dalam fungsi (<i>nested function</i>) khusus untuk kalkulasi jarak.
16	<code>def hitung_jarak(titik1, titik2):</code>	Mendefinisikan fungsi lokal bernama <code>hitung_jarak</code> yang menerima dua buah parameter berupa objek titik koordinat.
17	<code>return math.sqrt((titik1[0] - titik2[0])**2 + (titik1[1] - titik2[1])**2 + (titik1[2] - titik2[2])**2)</code>	Menghitung rumus jarak Euclidean dimensi tiga dengan mengakses indeks elemen <i>tuple</i> (indeks 0 = x1, 1 = x2, 2 = x3) lalu mengembalikan hasil akarnya.

18	# Hitung jarak dari U ke setiap cluster	Komentar dokumentasi untuk menjelaskan alur bahwa program mulai mengeksekusi perhitungan jarak dari posisi titik U ke seluruh pusat cluster.
19	$U = (x1, x2, x3)$	Menyatukan nilai parameter input variabel x1, x2, x3 ke dalam satu kesatuan objek <i>tuple</i> baru bernama U.
20	jarak_A = hitung_jarak(U, A)	Memanggil fungsi internal hitung_jarak untuk mencari jarak titik U ke pusat cluster A, lalu disimpan ke dalam variabel baru.
21	jarak_B = hitung_jarak(U, B)	Memanggil fungsi internal hitung_jarak untuk mencari jarak titik U ke pusat cluster B, lalu disimpan ke dalam variabel baru.
22	jarak_C = hitung_jarak(U, C)	Memanggil fungsi internal hitung_jarak untuk mencari jarak titik U ke pusat cluster C, lalu disimpan ke dalam variabel baru.
23	# Cetak informasi titik dan jarak (Gunakan format .7f agar presisi desimalnya terlihat)	Komentar dokumentasi untuk memberikan instruksi bahwa output pencetakan jarak wajib menggunakan format desimal tinggi agar hasil pecahan terlihat jelas.
24	print(f'Menguji Titik U: ({x1:.7f}, {x2:.7f}, {x3:.7f})')	Mencetak koordinat titik U yang diuji ke layar dengan format presisi tinggi tingkat desimal (.7f).
25	print(f'- Jarak ke Cluster A: {jarak_A:.4f}')	Mencetak hasil kalkulasi jarak ke cluster A dengan pembatasan pembulatan rapi sepanjang 4 angka di belakang desimal (.4f).
26	print(f'- Jarak ke Cluster B: {jarak_B:.4f}')	Mencetak hasil kalkulasi jarak ke cluster B dengan pembatasan pembulatan rapi sepanjang 4 angka di belakang desimal (.4f).
27	print(f'- Jarak ke Cluster C: {jarak_C:.4f}')	Mencetak hasil kalkulasi jarak ke cluster C dengan pembatasan pembulatan rapi sepanjang 4 angka di belakang desimal (.4f).
28	# KONDISI 2: Kondisi Khusus (Deteksi Cluster yang Jaraknya Sama)	Komentar dokumentasi yang menandai awal masuknya sistem ke logika percabangan untuk mendeteksi kasus posisi titik seimbang (tie-breaker).
29	# Ditambahkan pembulatan mikro (round) agar toleransi limit angka pecahan di komputer aman	Komentar dokumentasi yang menjelaskan alasan teknis penggunaan fungsi round(), yaitu untuk menyeragamkan batas limit memori angka pecahan komputer agar terhindar dari <i>floating-point error</i> .

30	<code>jA, jB, jC = round(jarak_A, 7), round(jarak_B, 7), round(jarak_C, 7)</code>	Melakukan pembulatan mikro nilai variabel jarak hingga batas 7 angka di belakang desimal untuk keperluan komparasi logika yang akurat.
31	<code>if jA == jB and jB == jC:</code>	Mengevaluasi kondisi khusus pertama dengan memeriksa apakah posisi titik U memiliki jarak terdekat yang persis sama kuat ke tiga cluster sekaligus.
32	<code>print("Hasil: KONDISI KHUSUS (Titik U berada tepat di tengah-tengah antara Cluster A, B, dan C)\n")</code>	Mencetak Hasil: KONDISI KHUSUS (Titik U berada tepat di tengah-tengah antara Cluster A, B, dan C).
33	<code>elif jA == jB and jA <= jC:</code>	Mengevaluasi variasi kondisi khusus lainnya di mana titik U berada di posisi seimbang di antara dua cluster terdekat yang bernilai kembar, yaitu cluster A dan B.
34	<code>print("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER B)\n")</code>	Mencetak Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER B).
35	<code>elif jA == jC and jA <= jB:</code>	Mengevaluasi variasi kondisi khusus lainnya di mana titik U berada di posisi seimbang di antara dua cluster terdekat yang bernilai kembar, yaitu cluster A dan C.
36	<code>print("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER C)\n")</code>	Mencetak Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER C).
37	<code>elif jB == jC and jB <= jA:</code>	Mengevaluasi variasi kondisi khusus lainnya di mana titik U berada di posisi seimbang di antara dua cluster terdekat yang bernilai kembar, yaitu cluster B dan C.
38	<code>print("Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER B dan CLUSTER C)\n")</code>	Mencetak Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER B dan CLUSTER C).
39	<code># KONDISI 3: Kondisi Normal (Satu Cluster Terdekat)</code>	Komentar dokumentasi yang menandai bagian blok alternatif percabangan jika kondisi khusus (tie-break) tidak terpenuhi (posisi jarak normal).
40	<code>elif jA < jB and jA < jC:</code>	Mengevaluasi jika U tidak termasuk kondisi khusus dan input tervalidasi int atau float, selanjutnya nilai jarak A lebih kecil dari nilai jarak B dan nilai jarak C lakukan perintah pada baris setelah ini.

41	<code>print("Hasil: KONDISI NORMAL (CLUSTER A)\n")</code>	Mencetak Hasil: KONDISI NORMAL (CLUSTER A).
42	<code>elif jB < jC:</code>	Mengevaluasi jika nilai jarak A tidak paling kecil dan nilai jarak B lebih kecil dari nilai jarak C lakukan perintah pada baris setelah ini.
43	<code>print("Hasil: KONDISI NORMAL (CLUSTER B)\n")</code>	Mencetak Hasil: KONDISI NORMAL (CLUSTER B).
44	<code>else:</code>	Mengevaluasi jika nilai jarak C yang paling kecil.
45	<code>print("Hasil: KONDISI NORMAL (CLUSTER C)\n")</code>	Mencetak Hasil: KONDISI NORMAL (CLUSTER C).
46	<code># UJI COBA</code>	Komentar dokumentasi pengujian fungsi dengan berbagai kasus data.
47	<code>tentukan_cluster(0, 2, 0.5)</code>	Memanggil fungsi untuk kondisi khusus dengan U (0, 2, 0.5).
48	<code>tentukan_cluster(-1, 2, -1)</code>	Memanggil fungsi untuk kondisi normal dengan U (-1, 2, -1).
49	<code>tentukan_cluster("Lanang", 3, 5)</code>	Memanggil fungsi untuk kondisi input error dengan U ("Lanang", 3, 5).

5.6 Penjelasan Line-By-Line Code Program 6

Bagian terakhir ini menjabarkan instruksi per baris untuk Program 6. Penjelasan meliputi implementasi pada bahasa pemrograman R dan Python.

5.6.1 Penjelasan Line-By-Line Code R

Berikut adalah penjelasan instruksi Program 6 yang ditulis dalam *sintaks* R:

No.	Baris Setiap Kode	Penjelasan
1	<code>#PROGRAM NO 6</code>	Menandai bahwa ini adalah kode untuk program nomor 6
2	<code>hitung_interval_konfidensi <- function(p_hat,alpha,n) {</code>	Mendeklarasikan fungsi baru yang bernama <code>hitung_interval_konfidensi</code> yang menerima 3 parameter yaitu <code>p_hat</code> (proporsi), <code>alpha</code> (taraf signifikansi), dan <code>n</code> (jumlah sampel)
3	<code>if (p_hat < 0 p_hat > 1) {</code>	Membuat kondisi apabila <code>p_hat</code> kurang dari 0 atau proporsi lebih dari 1
4	<code>cat("Proporsi (p_hat) harus berada di antara 0 dan 1.\n")</code>	Mencetak sebuah pesan kesalahan apabila kondisi pada baris 3 terpenuhi
5	<code>} else if (n <= 0) {</code>	Menambahkan kondisi apabila <code>n</code> kurang dari atau sama dengan 0
6	<code>cat("Ukuran sampel (n) harus lebih besar dari 0.\n")</code>	Mencetak sebuah pesan kesalahan apabila kondisi pada baris 5 terpenuhi

7	<code>} else if (!(alpha == 0.10 alpha == 0.05)) {</code>	Menambahkan kondisi apabila alpha tidak sama dengan 0,10 atau 0,05
8	<code>cat("Nilai alpha harus 0.10 atau 0.05.\n")</code>	Mencetak sebuah pesan kesalahan apabila kondisi pada baris 7 terpenuhi
9	<code>} else {</code>	Menambahkan kondisi apabila kondisi dari baris 3,5, dan 7 tidak terpenuhi
10	<code># Jika semua valid, baru hitung</code>	Menandai bahwa proses perhitungan dimulai dari titik ini.
11	<code>z <- ifelse(alpha == 0.10, 1.645, 1.96)</code>	Menentukan nilai z, apabila alpha bernilai 0.10 maka z bernilai 1,645, jika tidak maka z bernilai 1,96
12	<code>margin_of_error <- z * sqrt((p_hat * (1 - p_hat)) / n)</code>	Menghitung nilai <i>Margin of Error</i>
13	<code>batas_bawah <- p_hat - margin_of_error</code>	Menghitung nilai batas bawah
14	<code>batas_atas <- p_hat + margin_of_error</code>	Menghitung nilai batas atas
15	<code>cat("\n--- Hasil Perhitungan ---\n")</code>	Mencetak teks dekoratif sebagai judul atau pembatas output hasil perhitungan di konsol.
16	<code>cat(sprintf("\nInterval Konfidensi (%.0f%%): %.4f < p < %.4f\n", (1 - alpha) * 100, batas_bawah, batas_atas))</code>	Menggunakan fungsi <code>sprintf</code> untuk memformat teks output. <code>%.0f%%</code> akan menampilkan persentase tingkat kepercayaan tanpa desimal, dan <code>%.4f</code> memformat batas interval menjadi 4 angka di belakang koma.
17	<code>(1 - alpha) * 100, batas_bawah, batas_atas))</code>	Mengubah alpha menjadi persen tingkat kepercayaan (misal 95%), serta memasukkan nilai <code>batas_bawah</code> dan <code>batas_atas</code>
18	<code>}</code>	Penutup blok kode else (akhir dari proses perhitungan utama).
19	<code>}</code>	Menutup deklarasi dari fungsi hitung interval konfidensi
20	<code># Menjalankan FUNGSI</code>	Menandai bagian eksekusi atau pengujian fungsi dengan berbagai kasus data.
21	<code>hitung_interval_konfidensi(0.5,0.05,100)</code>	Memanggil fungsi untuk kasus normal dengan <code>p_hat = 0,5</code> , <code>alpha = 0,05</code> , dan <code>n = 100</code>
22	<code>hitung_interval_konfidensi(0.5,0.1,100)</code>	Memanggil fungsi untuk kasus normal dengan <code>p_hat = 0,5</code> , <code>alpha = 0,1</code> , dan <code>n = 100</code>
23	<code>hitung_interval_konfidensi(0.5,0.2,100)</code>	Memanggil fungsi untuk validasi error dengan <code>p_hat = 0,5</code> , <code>alpha = 0,2</code> , dan <code>n = 100</code>
24	<code>hitung_interval_konfidensi(1.2,0.1,100)</code>	Memanggil fungsi untuk validasi error dengan <code>p_hat = 1,2</code> , <code>alpha = 0,1</code> , dan <code>n = 100</code>

5.6.2 Penjelasan Line-By-Line Code Python

Adapun penjelasan instruksi Program 6 yang ditulis dalam *sintaks* Python adalah sebagai berikut:

No.	Baris Kode Python	Penjelasan
1	<code>import math</code>	Mengimpor modul bawaan math untuk menggunakan fungsi matematika, khusus nya operasi akar kuadrat
2	<code>def hitung_interval_konfidensi(p_hat, n, alpha):</code>	Mendefinisikan fungsi yang bernama hitung_interval_konfidensi dengan 3 parameter, yaitu p_hat (proporsi), n (jumlah sampel), dan alpha (taraf signifikansi)
3	<code>try:</code>	Menguji kode untuk mencari error
4	<code># Konversi input ke tipe data yang sesuai</code>	Menjelaskan penyesuaian tipe data setiap parameter
5	<code>p_hat = float(p_hat)</code>	Menyesuaikan variabel p_hat menjadi tipe data float (bilangan desimal)
6	<code>n = int(n)</code>	Menyesuaikan variabel n menjadi tipe data integer (bilangan bulat)
7	<code>alpha = float(alpha)</code>	Menyesuaikan variabel alpha menjadi tipe data float (bilangan desimal)
8	<code>print("--- Program Interval Konfidensi ---")</code>	Mencetak string sebagai tanda program dimulai
9	<code># VALIDASI INPUT</code>	Menandai bahwa baris-baris dibawahnya berfungsi untuk mengecek validasi data input
10	<code>if p_hat < 0 or p_hat > 1:</code>	Membuat kondisi apabila p_hat kurang dari 0 atau proporsi lebih dari 1
11	<code>print("Error: Proporsi (p_hat) harus berada di antara 0 dan 1.")</code>	Mencetak sebuah pesan kesalahan apabila nilai p_hat kurang dari 0 atau lebih dari 1
12	<code>elif n <= 0:</code>	Menambahkan kondisi apabila n kurang dari atau sama dengan 0
13	<code>print("Error: Ukuran sampel (n) harus lebih besar dari 0.")</code>	Mencetak sebuah pesan kesalahan apabila nilai n kurang dari atau sama dengan 0
14	<code>elif alpha != 0.10 and alpha != 0.05:</code>	Menambahkan kondisi apabila alpha tidak sama dengan 0,10 atau alpha tidak sama dengan 0,05
15	<code>print("Error: Nilai alpha harus 0.10 atau 0.05.")</code>	Mencetak sebuah kesalahan apabila alpha tidak sama 0,10 atau alpha tidak sama dengan 0,05
16	<code>else:</code>	Menambahkan kondisi apabila seluruh rangkaian validasi kondisi tidak terpenuhi
17	<code># Penentuan nilai z</code>	Menandai bahwa baris selanjutnya adalah penentuan nilai z

18	<code>z = 1.645 if alpha == 0.10 else 1.96</code>	Memberikan nilai z sebesar 1,645 jika alpha bernilai 0,10 dan memberikan nilai z sebesar 1,96 jika alpha bernilai 0,05
19	<code># Perhitungan Margin of Error</code>	Menandai kode untuk perhitungan margin of error
20	<code>margin_of_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)</code>	Menghitung nilai Margin of error
21	<code>batas_bawah = p_hat - margin_of_error</code>	Menghitung nilai batas bawah
22	<code>batas_atas = p_hat + margin_of_error</code>	Menghitung nilai batas atas
23	<code># Output hasil</code>	Menandai bagian dari hasil perhitungan
24	<code>print("\n--- Hasil Perhitungan ---")</code>	Mencetak string sebagai tanda hasil perhitungan
25	<code>print(f"Interval Konfidensi ({(1 - alpha) * 100:.0f}%): "</code>	Mencetak bagian pertama teks hasil menggunakan f-string. Ekspresi $(1 - \alpha) * 100$ mengubah nilai desimal alpha menjadi persentase tingkat kepercayaan bulat (tanpa desimal).
26	<code>f'{batas_bawah:.4f} < p < {batas_atas:.4f}')</code>	Menyambung baris cetakan sebelumnya untuk menampilkan nilai batas_bawah dan batas_atas yang dibatasi tepat 4 angka di belakang koma (:.4f).
27	<code>except ValueError:</code>	Menangani jika ada kesalahan dalam menginput
28	<code># Menangani jika user memasukkan teks (bukan angka)</code>	Menjelaskan tujuan dari penanganan eksepsi ValueError.
29	<code>print("Error: Input harus berupa angka yang valid.")</code>	Mencetak pesan kesalahan untuk menginput data berupa angka
30	<code># Jalankan program</code>	Menandai bagian pengujian program
31	<code>if __name__ == "__main__":</code>	Memastikan bahwa baris kode di bawahnya hanya dieksekusi jika file ini dijalankan secara langsung (bukan di-import sebagai modul oleh file lain).
32	<code>hitung_interval_konfidensi(0.5,100,0. 05)</code>	Menguji fungsi untuk kasus normal dengan $p_{\text{hat}} = 0,5$, $\alpha = 0,05$, dan $n = 100$
33	<code>hitung_interval_konfidensi(0.5,100,0. 1)</code>	Menguji fungsi untuk kasus normal dengan $p_{\text{hat}} = 0,5$, $\alpha = 0,1$, dan $n = 100$
34	<code>hitung_interval_konfidensi(0.5,100,0. 2)</code>	Menguji fungsi untuk validasi error dengan $p_{\text{hat}} = 0,5$, $\alpha = 0,2$, dan $n = 100$
35	<code>hitung_interval_konfidensi(1.2,100,0. 05)</code>	Menguji fungsi untuk validasi error dengan $p_{\text{hat}} = 1,2$, $\alpha = 0,1$, dan $n = 100$

BAB VI

PENGUJIAN PROGRAM

6.1 Pengujian Program 1

Bagian ini menjabarkan hasil pengujian yang telah dilakukan pada Program 1. Pengujian ini bertujuan untuk memastikan program berjalan dan berfungsi dengan baik.

6.1.1 Tabel Pengujian

Tabel di bawah ini menyajikan skenario pengujian, input data, ekspektasi *output*, serta status keberhasilan untuk Program 1:

No.	Skenario Pengujian	Input Data	Ekspektasi Output Data	Status Pengujian
1	Pengujian kasus untuk mencetak kalimat biasa.	Ibu sedang masak di dapur, sedangkan ayah mencuci motor. Kucing tidur di atas kasur, sementara burung bernyanyi di luar sangkar.	Program dapat menampilkan <i>output</i> berupa banyak kalimat dan kata.	Berhasil
2	Pengujian kasus untuk mencetak kalimat yang berisi angka.	Indonesia merdeka pada tanggal 17 Agustus 1945.	Program dapat menampilkan <i>output</i> berupa banyak kalimat dan kata.	Berhasil
3	Pengujian kasus untuk mencetak kalimat yang berisi angka/harga.	Harga buku ini 50000 namun aku hanya membawa 30000.	Program dapat menampilkan <i>output</i> berupa banyak kalimat dan kata.	Berhasil

6.1.2. Screenshot Hasil Running

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan hasil eksekusi Program 1 berdasarkan skenario pengujian yang telah dilakukan:

- a. *Screenshot* pada R

Tampilan hasil keluaran Program 1 pada lingkungan eksekusi R:

```
> #Program 1
> #Kondisi 1
> teks <- "Ibu sedang masak di dapur, sedangkan ayah mencuci motor. Kucing tidur di atas kasur, sementara burung bernyanyi di luar sangkar."
> jumlahKalimat <- 0
> for(i in 1:nchar(teks)){
+   if(substr(teks, i, i) == "."){
+     jumlahKalimat <- jumlahKalimat + 1
+   }
+ }
>
> kata <- strsplit(teks, " ")
> jumlahKata <- length(kata[[1]])
>
> cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata, "kata")
Teks tersebut memuat 1 kalimat dan 7 kata>
> #Kondisi 2
> teks <- "Indonesia merdeka pada tanggal 17 Agustus 1945."
>
> jumlahKalimat <- 0
> for(i in 1:nchar(teks)){
+   if(substr(teks, i, i) == "."){
+     jumlahKalimat <- jumlahKalimat + 1
+   }
+ }
>
> kata <- strsplit(teks, " ")
> jumlahKata <- length(kata[[1]])
>
> cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata, "kata")
Teks tersebut memuat 1 kalimat dan 7 kata>
> #Kondisi 3
> teks <- "Harga buku ini 50000 namun aku hanya membawa 30000."
>
> jumlahKalimat <- 0
> for(i in 1:nchar(teks)){
+   if(substr(teks, i, i) == "."){
+     jumlahKalimat <- jumlahKalimat + 1
+   }
+ }
>
> kata <- strsplit(teks, " ")
> jumlahKata <- length(kata[[1]])
>
> cat("Teks tersebut memuat", jumlahKalimat, "kalimat dan", jumlahKata, "kata")
Teks tersebut memuat 1 kalimat dan 9 kata>
```

b. Screenshot pada Phyton

Tampilan hasil keluaran Program 1 pada lingkungan eksekusi Python:

```
*** Masukkan Teks:Ibu sedang masak di dapur, sedangkan ayah mencuci motor. Kucing tidur di atas kasur, sementara burung bernyanyi di luar sangka
Teks tersebut memuat 2 kalimat dan 20 kata.

Masukkan Teks:Indonesia merdeka pada tanggal 17 Agustus 1945.
Teks tersebut memuat 1 kalimat dan 7 kata.

*** Masukkan Teks:Harga buku ini 50000 namun aku hanya membawa 30000.
Teks tersebut memuat 1 kalimat dan 9 kata.
```

6.1.3. Interpretasi *Output*

Uraian berikut menjelaskan hasil keluaran dari Program 1 berdasarkan skenario pengujian yang telah dilakukan:

- Berdasarkan input yang diberikan pada skenario pertama, diperoleh *output* teks memuat 2 kalimat dan 20 kata. Baik logika perulangan karakter pada Python maupun kombinasi fungsi `substr()` dan `strsplit()` pada R mampu membedakan tanda koma (diabaikan) dan hanya mengakumulasi tanda titik sebagai pembatas kalimat. Hasil pemisahan kata berbasis spasi juga terbukti konsisten menghasilkan panjang vektor 20 elemen kata.

- b. Berdasarkan input yang diberikan pada skenario kedua, program menghasilkan perhitungan 1 kalimat dan 7 kata. Angka pecahan tahun seperti “1945.” di akhir kalimat berhasil dibaca secara tepat sebagai satu kesatuan kata sebelum dipotong oleh pembatas spasi, dan karakter titiknya sukses memicu bertambahnya variabel kalimat.
- c. Berdasarkan input yang diberikan pada skenario ketiga, program membuktikan bahwa algoritma ini memperlakukan karakter numerik sama seperti karakter alfabet. Angka nominal tersebut tetap dihitung sebagai satu kata utuh karena dibatasi oleh spasi di kanan dan kirinya. Program sukses mengeluarkan *output* 1 kalimat dan 9 kata secara valid.

6.2 Pengujian Program 2

Bagian ini menjabarkan hasil pengujian yang telah dilakukan pada Program 2. Pengujian ini bertujuan untuk memastikan program berjalan dan berfungsi dengan baik.

6.2.1 Tabel Pengujian

Tabel di bawah ini menyajikan skenario pengujian, input data, ekspektasi *output*, serta status keberhasilan untuk Program 2:

No.	Skenario Pengujian	Input Data	Ekspektasi Output Data	Status Pengujian
1	Pengujian kasus untuk mencetak <i>string</i> yang memuat tanda petik tunggal di dalamnya.	Tidak ada input manual (menggunkan nilai inisialisasi pada variabel K1).	Program dapat menampilkan <i>output</i> berupa kalimat: Saya tak 'kan menyerah.	Berhasil
2	Pengujian kasus untuk mencetak <i>string</i> yang memuat kutipan kalimat langsung dengan tanda petik ganda di dalamnya.	Tidak ada input manual (menggunkan nilai inisialisasi pada variabel K2).	Program dapat menampilkan <i>output</i> berupa kalimat: Ia berkata, “Aku menyayangimu.”	Berhasil
3	Pengujian kasus untuk mencetak <i>string</i> yang memuat kombinasi tanda petik tunggal dan ganda sekaligus, serta tambahan <i>escape character</i> .	Tidak ada input manual (menggunkan nilai inisialisasi pada variabel K3).	Program dapat menampilkan <i>output</i> berupa kalimat: “Coba jelaskan pengertian ‘cross-validation’ dalam Machine Learning!”	Berhasil
4	Pengujian kasus untuk mencetak <i>string</i> yang memuat	Tidak ada input manual (menggunkan	Program dapat menampilkan <i>output</i> berupa kalimat:	Berhasil

	kombinasi huruf, angka, dan karakter tanda baca.	nilai inisialisasi pada variabel K4).	Surat keputusan itu bernomor 62/UN.34/19/2023.	
--	--	---------------------------------------	--	--

6.2.2. Screenshot Hasil *Running*

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan hasil eksekusi Program 2 berdasarkan skenario pengujian yang telah dilakukan:

a. *Screenshot* pada R

Tampilan hasil keluaran Program 2 pada lingkungan eksekusi R:

```
> #Program String_Kalimat
> K1 <- "Saya tak 'kan menyerah."
> K2 <- 'Ia berkata, "Aku menyayangimu."'
> K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
> K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."
> cat(K1, "\n")
Saya tak 'kan menyerah.
> cat(K2, "\n")
Ia berkata, "Aku menyayangimu."
> cat(K3, "\n")
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
> cat(K4, "\n")
Surat keputusan itu bernomor 62/UN.34/19/2023.
```

b. *Screenshot* pada Python

Tampilan hasil keluaran Program 2 pada lingkungan eksekusi Python:

```
... Saya tak 'kan menyerah.
    Ia berkata, "Aku menyayangimu."
    "Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
    Surat keputusan itu bernomor 62/UN.34/19/2023.
```

6.2.3. Interpretasi *Output*

Uraian berikut menjelaskan hasil keluaran dari Program 2 berdasarkan skenario pengujian yang telah dilakukan:

- Berdasarkan pengeksekusian kode pada skenario pertama, dapat diperoleh *output* berupa kalimat “Saya tak 'kan menyerah.” yang membuktikan bahwa program berhasil membaca dan mencetak karakter petik tunggal di dalam *string* tanpa menyebabkan *error sintaks*.
- Berdasarkan pengeksekusian kode pada skenario kedua, dapat diperoleh *output* berupa kalimat ‘Ia berkata, “Aku menyayangimu.”’ yang membuktikan bahwa program berhasil mencetak karakter tanda petik ganda (sebagai kutipan kalimat langsung) di dalam pembatas tanda petik tunggal secara utuh.
- Berdasarkan pengeksekusian kode pada skenario ketiga, dapat diperoleh *output* berupa kalimat ““Coba Jelaskan pengertian ‘cross-validation’ dalam Machine Learning!”” yang menunjukkan keberhasilan program dalam memproses penggunaan *escape character* (\) untuk mencetak tanda petik ganda dan tunggal secara bersamaan di dalam satu *string*.

- d. Berdasarkan pengekseskusion kode pada skenario keempat, dapat diperoleh *output* berupa kalimat “Surat keputusan itu bernomor 62/UN.34/19/2023.” yang membuktikan bahwa program mampu memproses dan mencetak kombinasi huruf, angka, dan karakter tanda baca dengan format yang tepat.

6.3 Pengujian Program 3

Bagian ini menjabarkan hasil pengujian yang telah dilakukan pada Program 3. Pengujian ini bertujuan untuk memastikan program berjalan dan berfungsi dengan baik.

6.3.1 Tabel Pengujian

Tabel di bawah ini menyajikan skenario pengujian, input data, ekspektasi *output*, serta status keberhasilan untuk Program 3:

No.	Skenario Pengujian	Input Data	Ekspektasi Output Data	Status Pengujian
1	Pengujian kasus untuk mencetak hasil dari $D=0$	A = 1 B = 2 C = 1	Program dapat menampilkan <i>output</i> berupa hasil akar kembar	Berhasil
2	Pengujian kasus untuk mencetak hasil dari $D<0$	A = 1 B = -5 C = 6	Program dapat menampilkan <i>output</i> berupa hasil akar real	Berhasil
3	Pengujian kasus untuk mencetak hasil dari $D>0$	A = 1 B = 2 C = 5	Program dapat menampilkan <i>output</i> berupa kalimat “Persamaan hanya memiliki akar imajiner”	Berhasil

6.3.2. Screenshot Hasil Running

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan hasil eksekusi Program 3 berdasarkan skenario pengujian yang telah dilakukan:

- a. *Screenshot* pada R

Tampilan hasil keluaran Program 3 pada lingkungan eksekusi R:

```

> #Program 3
> #Kondisi 1
> a <- 1
> b <- 2
> c <- 1
>
> D <- b^2 - 4*a*c
>
> if (D == 0) {
+   x <- -b / (2*a)
+   cat("Nilai x kembar =", round(x, 3), "\n")
+ } else if (D > 0) {
+   x1 <- (-b + sqrt(D)) / (2*a)
+   x2 <- (-b - sqrt(D)) / (2*a)
+   cat("Akar real:\n")
+   cat("x1 =", round(x1, 3), "\n")
+   cat("x2 =", round(x2, 3), "\n")
+ } else {
+   cat("Persamaan hanya memiliki akar imajiner\n")
+ }
Nilai x kembar = -1

> #Kondisi 2
> a <- 1
> b <- -5
> c <- 6
>
> D <- b^2 - 4*a*c
>
> if (D == 0) {
+   x <- -b / (2*a)
+   cat("Nilai x kembar =", round(x, 3), "\n")
+ } else if (D > 0) {
+   x1 <- (-b + sqrt(D)) / (2*a)
+   x2 <- (-b - sqrt(D)) / (2*a)
+   cat("Akar real:\n")
+   cat("x1 =", round(x1, 3), "\n")
+   cat("x2 =", round(x2, 3), "\n")
+ } else {
+   cat("Persamaan hanya memiliki akar imajiner\n")
+ }
Akar real:
x1 = 3
x2 = 2
>

> #Kondisi 3
> a <- 1
> b <- 2
> c <- 5
>
> D <- b^2 - 4*a*c
>
> if (D == 0) {
+   x <- -b / (2*a)
+   cat("Nilai x kembar =", round(x, 3), "\n")
+ } else if (D > 0) {
+   x1 <- (-b + sqrt(D)) / (2*a)
+   x2 <- (-b - sqrt(D)) / (2*a)
+   cat("Akar real:\n")
+   cat("x1 =", round(x1, 3), "\n")
+   cat("x2 =", round(x2, 3), "\n")
+ } else {
+   cat("Persamaan hanya memiliki akar imajiner\n")
+ }
Persamaan hanya memiliki akar imajiner
>

```

b. *Screenshot* pada Python

Tampilan hasil keluaran Program 3 pada lingkungan eksekusi Python:

```

*** Masukkan nilai a: 1
    Masukkan nilai b: 2
    Masukkan nilai c: 1
    Nilai x kembar = -1.0

*** Masukkan nilai a: 1
    Masukkan nilai b: -5
    Masukkan nilai c: 6
    Akar real:
    x1 = 3.0
    x2 = 2.0

*** Masukkan nilai a: 1
    Masukkan nilai b: 2
    Masukkan nilai c: 5
    Persamaan hanya memiliki akar imajiner

```

6.3.3. Interpretasi *Output*

Uraian berikut menjelaskan hasil keluaran dari Program 3 berdasarkan skenario pengujian yang telah dilakukan:

- Berdasarkan input yang diberikan pada skenario pertama, dapat diperoleh *output* berupa akar kembar dari hasil yang diperoleh, yaitu $x = -1$.
- Berdasarkan input yang diberikan pada skenario kedua, dapat diperoleh *output* berupa akar real dari hasil yang diperoleh, yaitu $x_1 = 3$ dan $x_2 = 2$.
- Berdasarkan input yang diberikan pada skenario ketiga, dapat diperoleh *output* kalimat “Persamaan hanya memiliki akar imajiner”.

6.4 Pengujian Program 4

Bagian ini menjabarkan hasil pengujian yang telah dilakukan pada Program 4. Pengujian ini bertujuan untuk memastikan program berjalan dan berfungsi dengan baik.

6.4.1 Tabel Pengujian

Tabel di bawah ini menyajikan skenario pengujian, input data, ekspektasi *output*, serta status keberhasilan untuk Program 4:

No.	Skenario Pengujian	Input Data	Ekspektasi Output Data	Status Pengujian
1	Pengujian kasus untuk input normal dengan kombinasi digit tanggal yang valid.	NIP = 19941021 2019031010	Program akan menghasilkan output berupa teks: Tanggal lahir Anda: 21 Oktober 1994	Berhasil
2	Pengujian kasus untuk input tanggal kabisat (29 Februari) pada tahun yang memenuhi kriteria tahun kabisat.	NIP = 20000229 2022011002	Program akan menghasilkan output berupa teks: Tanggal lahir Anda: 29 Februari 2000	Berhasil
3	Pengujian kasus untuk input tanggal 29 Februari pada tahun yang tidak memenuhi	NIP = 20030229 2024011004	Program akan menghasilkan output berupa peringatan:	Berhasil

	kriteria tahun kabisat atau dengan kata lahir melebihi batas maksimal hari.		NIP tidak valid, harap masukkan ulang.	
4	Pengujian kasus lanjutan untuk menguji fungsi perulangan dengan memasukkan input tanggal yang valid setelah penolakan sebelumnya.	NIP = 20030228 2024011004	Program akan menghasilkan output berupa teks: Tanggal lahir Anda: 28 Februari 2003	Berhasil

6.4.2. Screenshot Hasil Running

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan hasil eksekusi Program 4 berdasarkan skenario pengujian yang telah dilakukan:

a. Screenshot pada R

Tampilan hasil keluaran Program 4 pada lingkungan eksekusi R:

```
> #Program NIP_ASN
> #Skenario 1
> #Inisialisasi kamus bulan dan batas hari
> data_bulan <- c("Januari", "Februari", "Maret", "April", "Mei", "Juni",
+               "Juli", "Agustus", "September", "Oktober", "November", "Desember"
+ )
> data_hari <- c(31, 28, 31, 30, 31, 30,
+              31, 31, 30, 31, 30, 31
+ )
> while (TRUE) {
+   #Menerima input dari user
+   NIP <- readline(prompt = "Masukkan 18 digit NIP Anda: ")
+   tahun <- as.numeric(substr(NIP, 1, 4)) #Indeks R mulai dari 1
+   bulan <- as.numeric(substr(NIP, 5, 6))
+   tanggal <- as.numeric(substr(NIP, 7, 8))
+
+   #Validasi rentang bulan
+   if (bulan < 1 | bulan > 12){
+     cat("NIP tidak valid, harap masukkan ulang.\n\n")
+     next
+   }
+
+   #Mengambil data dari kamus
+   nama_bulan <- data_bulan[bulan]
+   max_hari <- data_hari[bulan]
+
+   #Cek tahun kabisat (update batas Februari)
+   if (bulan == 2 & (tahun %% 400 == 0 | (tahun %% 4 == 0 & tahun %% 100 != 0))){
+     max_hari = 29
+   }
+
+   #Validasi batas tanggal
+   if (tanggal < 1 | tanggal > max_hari){
+     cat("NIP tidak valid, harap masukkan ulang.\n\n")
+     next
+   }
+
+   #Jika semua valid
+   cat("Tanggal lahir Anda:", tanggal, nama_bulan, tahun, "\n")
+   break
+ }
Masukkan 18 digit NIP Anda: 199410212019031010
Tanggal lahir Anda: 21 Oktober 1994
>
> #Skenario 2
> #Inisialisasi kamus bulan dan batas hari
> data_bulan <- c("Januari", "Februari", "Maret", "April", "Mei", "Juni",
+               "Juli", "Agustus", "September", "Oktober", "November", "Desember"
+ )
> data_hari <- c(31, 28, 31, 30, 31, 30,
+              31, 31, 30, 31, 30, 31
+ )
```

```

> while (TRUE) {
+   #Menerima input dari user
+   NIP <- readline(prompt = "Masukkan 18 digit NIP Anda: ")
+   tahun <- as.numeric(substr(NIP, 1, 4)) #Indeks R mulai dari 1
+   bulan <- as.numeric(substr(NIP, 5, 6))
+   tanggal <- as.numeric(substr(NIP, 7, 8))
+
+   #Validasi rentang bulan
+   if (bulan < 1 | bulan > 12){
+     cat("NIP tidak valid, harap masukkan ulang.\n\n")
+     next
+   }
+
+   #Mengambil data dari kamus
+   nama_bulan <- data_bulan[bulan]
+   max_hari <- data_hari[bulan]
+
+   #Cek tahun kabisat (update batas Februari)
+   if (bulan == 2 & (tahun %% 400 == 0 | (tahun %% 4 == 0 & tahun %% 100 != 0))){
+     max_hari = 29
+   }
+
+   #Validasi batas tanggal
+   if (tanggal < 1 | tanggal > max_hari){
+     cat("NIP tidak valid, harap masukkan ulang.\n\n")
+     next
+   }
+
+   #Jika semua valid
+   cat("Tanggal lahir Anda:", tanggal, nama_bulan, tahun, "\n")
+   break
+ }
Masukkan 18 digit NIP Anda: 200002292022011002
Tanggal lahir Anda: 29 Februari 2000
>
> #Skenario 3
> #Inisialisasi kamus bulan dan batas hari
> data_bulan <- c("Januari", "Februari", "Maret", "April", "Mei", "Juni",
+   "Juli", "Agustus", "September", "Oktober", "November", "Desember"
+ )
> data_hari <- c(31, 28, 31, 30, 31, 30,
+   31, 31, 30, 31, 30, 31
+ )
> while (TRUE) {
+   #Menerima input dari user
+   NIP <- readline(prompt = "Masukkan 18 digit NIP Anda: ")
+   tahun <- as.numeric(substr(NIP, 1, 4)) #Indeks R mulai dari 1
+   bulan <- as.numeric(substr(NIP, 5, 6))
+   tanggal <- as.numeric(substr(NIP, 7, 8))
+
+   #Validasi rentang bulan
+   if (bulan < 1 | bulan > 12){
+     cat("NIP tidak valid, harap masukkan ulang.\n\n")
+     next
+   }
+
+   #Mengambil data dari kamus
+   nama_bulan <- data_bulan[bulan]
+   max_hari <- data_hari[bulan]
+
+   #Cek tahun kabisat (update batas Februari)
+   if (bulan == 2 & (tahun %% 400 == 0 | (tahun %% 4 == 0 & tahun %% 100 != 0))){
+     max_hari = 29
+   }
+
+   #Validasi batas tanggal
+   if (tanggal < 1 | tanggal > max_hari){
+     cat("NIP tidak valid, harap masukkan ulang.\n\n")
+     next
+   }
+
+   #Jika semua valid
+   cat("Tanggal lahir Anda:", tanggal, nama_bulan, tahun, "\n")
+   break
+ }
Masukkan 18 digit NIP Anda: 200302292024011004
NIP tidak valid, harap masukkan ulang.

Masukkan 18 digit NIP Anda: 200302282024011004
Tanggal lahir Anda: 28 Februari 2003

```

b. *Screenshot* pada Python

Tampilan hasil keluaran Program 4 pada lingkungan eksekusi Python:

```

*** Masukkan 18 digit NIP Anda:199410212019031010
    Tanggal lahir Anda: 21 Oktober 1994

```

```
*** Masukkan 18 digit NIP Anda:200002292022011002
    Tanggal lahir Anda: 29 Februari 2000

*** Masukkan 18 digit NIP Anda:200302292024011004
    NIP tidak valid, harap masukkan ulang.
    Masukkan 18 digit NIP Anda:200302282024011004
    Tanggal lahir Anda: 28 Februari 2003
```

6.4.3. Interpretasi *Output*

Uraian berikut menjelaskan hasil keluaran dari Program 4 berdasarkan skenario pengujian yang telah dilakukan:

- a. Berdasarkan input yang diberikan pada skenario pertama, dapat diperoleh *output* berupa teks “Tanggal lahir Anda: 21 Oktober 1994” yang menunjukkan bahwa program berhasil mengekstrak urutan digit tanggal, mengonversi kode angka bulan menjadi teks nama bulan, dan menyusunnya menjadi format penanggalan dengan tepat.
- b. Berdasarkan input yang diberikan pada skenario kedua, dapat diperoleh *output* berupa teks “Tanggal lahir Anda: 29 Februari 2000” yang membuktikan bahwa logika validasi kabisat pada program berfungsi dengan akurat, di mana tahun 2000 benar merupakan tahun kabisat dan program mampu memproses serta mengubah batas maksimal hari pada bulan Februari di tahun tersebut menjadi 29 hari.
- c. Berdasarkan input yang diberikan pada skenario ketiga, dapat diperoleh *output* sebuah peringatan “NIP tidak valid, harap masukkan ulang.” yang membuktikan bahwa program berhasil menolak input tanggal 29 Februari pada tahun yang bukan merupakan tahun kabisat (2003). Hal ini menunjukkan bahwa logika program dalam memvalidasi nilai tanggal yang melebihi batas maksimal hari berjalan dengan baik. sekaligus membuat perulangan agar pengguna memasukkan data kembali.
- d. Sebagai kelanjutan dari skenario ketiga, saat pengguna memberikan input perbaikan dengan mengganti tanggal menjadi 28, dapat diperoleh *output* berupa teks “Tanggal lahir Anda: 28 Februari 2003”. Hal ini membuktikan bahwa instruksi perulangan di dalam program dieksekusi dengan baik, di mana program tidak langsung berhenti saat terjadi penolakan, melainkan mampu merespons input baru dari pengguna dan memprosesnya hingga menghasilkan keluaran yang valid.

6.5 Pengujian Program 5

Bagian ini menjabarkan hasil pengujian yang telah dilakukan pada Program 5. Pengujian ini bertujuan untuk memastikan program berjalan dan berfungsi dengan baik.

6.5.1. Tabel pengujian

Tabel di bawah ini menyajikan skenario pengujian, input data, ekspektasi *output*, serta status keberhasilan untuk Program 5:

No.	Skenario Pengujian	Input Data	Ekspektasi Output Data	Status Pengujian
1	Pengujian kasus untuk input titik koordinat yang memiliki jarak terdekat yang sama antara dua cluster (Kondisi Khusus).	Titik U = (0, 2, 0.5)	Program menghitung jarak Euclidean ke tiga pusat cluster (A = 3.3541, B = 8.2006, C = 3.3541), mendeteksi kesamaan nilai terkecil, dan menampilkan pesan: "Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER C)"	Berhasil
2	Pengujian kasus untuk input titik koordinat normal dengan cluster terdekat tunggal pada Cluster C.	Titik U = (-1, 2, -1)	Program menghitung jarak Euclidean (A = 5.0990, B = 9.4340, C = 1.7321) dan menampilkan klaster terdekat secara mutlak melalui pesan: "Hasil: KONDISI NORMAL (CLUSTER C)"	Berhasil
3	Pengujian validasi input berupa data non-numerik (string) untuk menguji sistem penanganan kesalahan (<i>error handling</i>).	Titik U = ("Lanang", 3, 5)	Program mendeteksi kesalahan tipe data input dan langsung menghentikan proses kalkulasi matematis untuk mengeluarkan pesan peringatan: "Hasil: KONDISI ERROR (Input harus berupa angka!)"	Berhasil

6.5.2. Screenshot Hasil Running

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan hasil eksekusi Program 5 berdasarkan skenario pengujian yang telah dilakukan:

a. Screenshot pada R

Tampilan hasil keluaran Program 5 pada lingkungan eksekusi R:

```
> # UJI COBA KONDISI
> tentukan_cluster(0, 2, 0.5)
Menguji Titik U: (0.0, 2.0, 0.5)
- Jarak ke Cluster A: 3.3541
- Jarak ke Cluster B: 8.2006
- Jarak ke Cluster C: 3.3541
Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER C)

> tentukan_cluster(-1, 2, -1)
Menguji Titik U: (-1.0, 2.0, -1.0)
- Jarak ke Cluster A: 5.0990
- Jarak ke Cluster B: 9.4340
- Jarak ke Cluster C: 1.7321
Hasil: KONDISI NORMAL (CLUSTER C)

> tentukan_cluster("Lanang", 3, 5)
Menguji Titik U: (Lanang, 3, 5)
Hasil: KONDISI ERROR (Input harus berupa angka!)
```

b. Screenshot pada python

Tampilan hasil keluaran Program 3 pada lingkungan eksekusi Python:

```
... Menguji Titik U: (0.0000000, 2.0000000, 0.5000000)
- Jarak ke Cluster A: 3.3541
- Jarak ke Cluster B: 8.2006
- Jarak ke Cluster C: 3.3541
Hasil: KONDISI KHUSUS (Titik U memiliki jarak terdekat yang SAMA antara CLUSTER A dan CLUSTER C)

Menguji Titik U: (-1.0000000, 2.0000000, -1.0000000)
- Jarak ke Cluster A: 5.0990
- Jarak ke Cluster B: 9.4340
- Jarak ke Cluster C: 1.7321
Hasil: KONDISI NORMAL (CLUSTER C)

Menguji Titik U: (Lanang, 3, 5)
Hasil: KONDISI ERROR (Input harus berupa angka!)
```

6.5.3. Interpretasi Output

Uraian berikut menjelaskan hasil keluaran dari Program 5 berdasarkan skenario pengujian yang telah dilakukan:

- a. Berdasarkan input yang diberikan pada skenario pertama, dapat diperoleh output berupa nilai perhitungan jarak ke Cluster A sebesar 3.3541, Cluster B sebesar 8.2006, dan Cluster C sebesar 3.3541. Karena nilai jarak terkecil bernilai sama persis antara Cluster A dan Cluster C, program berhasil mengidentifikasi struktur data ini sebagai kondisi khusus. Hal ini membuktikan bahwa logika kondisional majemuk yang ditanamkan dalam kode mampu mendeteksi kondisi ambiguitas penentuan jarak klaster secara akurat..

- b. Berdasarkan input yang diberikan pada skenario kedua, diperoleh nilai perhitungan jarak matematis ke Cluster A sebesar 5.0990, Cluster B sebesar 9.4340, dan Cluster C sebesar 1.7321. Karena jarak minimum secara mutlak berada pada koordinat Cluster C, sistem mengambil keputusan yang valid dengan mengeluarkan status kondisi normal (cluster c).
- c. Berdasarkan input yang diberikan pada skenario keempat, sengaja diberikan anomali masukan berupa data string tekstual ("Lanang") pada elemen dimensi sumbu pertama. Program terbukti memiliki proteksi data yang matang (*robustness*) melalui mekanisme penangkapan eksepsi tipe data, sehingga alih-alih mengalami *force close* atau *crash*, program memberikan umpan balik informatif kepada pengguna berupa pesan kesalahan kondisi *error* (Input harus berupa angka!).

6.6 Pengujian Program 6

Bagian ini menjabarkan hasil pengujian yang telah dilakukan pada Program 6. Pengujian ini bertujuan untuk memastikan program berjalan dan berfungsi dengan baik.

6.6.1. Tabel pengujian

Tabel di bawah ini menyajikan skenario pengujian, input data, ekspektasi *output*, serta status keberhasilan untuk Program 6:

No.	Skenario Pengujian	Input Data	Ekspektasi Output Data	Status Pengujian
1	Pengujian kasus untuk input normal dengan nilai alpha = 0.05	$p_hat = 0.5$, $\alpha = 0.05$, $n = 100$	Program akan menghasilkan output berupa interval Konfidensi (95%): $0.4020 < p < 0.5980$	Berhasil
2	Pengujian kasus untuk input normal dengan nilai alpha = 0.1	$p_hat = 0.5$, $\alpha = 0.1$, $n = 100$	Program akan menghasilkan output berupa interval Konfidensi (90%): $0.4178 < p < 0.5823$	Berhasil
3	Pengujian kasus untuk input tidak valid yaitu nilai alpha tidak sama dengan 0.10 atau 0.05	$p_hat = 0.5$, $\alpha = 0.2$, $n = 100$	Program akan memberikan output berupa pesan kesalahan : Nilai alpha harus 0.10 atau 0.05	Berhasil

4	Pengujian kasus untuk input tidak valid yaitu nilai p_{hat} lebih besar dari 1	$p_{\text{hat}} = 1.2$, $\alpha = 0.1$, $n = 100$	Program akan memberikan output berupa pesan kesalahan : Proporsi (p_{hat}) harus berada di antara 0 dan 1	Berhasil
---	---	--	--	----------

6.6.2. Screenshot Hasil Running

Berikut adalah tangkapan layar (*screenshot*) yang menunjukkan hasil eksekusi Program 6 berdasarkan skenario pengujian yang telah dilakukan:

a. Screenshot pada R

Tampilan hasil keluaran Program 6 pada lingkungan eksekusi R:

```
> # Menjalankan FUNGSI
> hitung_interval_konfidensi(0.5,0.05,100)

--- Hasil Perhitungan ---

Interval Konfidensi (95%): 0.4020 < p < 0.5980
> hitung_interval_konfidensi(0.5,0.1,100)

--- Hasil Perhitungan ---

Interval Konfidensi (90%): 0.4178 < p < 0.5823
> hitung_interval_konfidensi(0.5,0.2,100)
Nilai alpha harus 0.10 atau 0.05.
> hitung_interval_konfidensi(1.2,0.1,100)
Proporsi (p_hat) harus berada di antara 0 dan 1.
> |
```

b. Screenshot pada Python

Tampilan hasil keluaran Program 6 pada lingkungan eksekusi Python:

```
... --- Program Interval Konfidensi ---

--- Hasil Perhitungan ---
Interval Konfidensi (95%): 0.4020 < p < 0.5980
--- Program Interval Konfidensi ---

--- Hasil Perhitungan ---
Interval Konfidensi (90%): 0.4178 < p < 0.5823
--- Program Interval Konfidensi ---
Error: Nilai alpha harus 0.10 atau 0.05.
--- Program Interval Konfidensi ---
Error: Proporsi (p_hat) harus berada di antara 0 dan 1.
```

6.6.3. Interpretasi *Output*

Uraian berikut menjelaskan hasil keluaran dari Program 6 berdasarkan skenario pengujian yang telah dilakukan:

- a. Berdasarkan input yang diberikan pada skenario pertama, dapat diperoleh *output* berupa nilai interval konfidensi dengan tingkat kepercayaan 95% berada pada rentang 0,4020 hingga 0,5980.
- b. Berdasarkan input yang diberikan pada skenario kedua, dapat diperoleh *output* berupa nilai interval konfidensi dengan tingkat kepercayaan 90% berada pada rentang 0,4178 hingga 0,5823.
- c. Berdasarkan input yang diberikan pada skenario ketiga, dapat diperoleh *output* sebuah teks yang berisi pesan kesalahan yaitu “Nilai alpha harus 0.10 atau 0.05.”. Hal ini menunjukkan bahwa validasi pada variabel alpha berjalan dengan baik untuk mencegah penggunaan nilai z yang tidak terdefinisi pada program.
- d. Berdasarkan input yang diberikan pada skenario keempat, dapat diperoleh *output* sebuah teks yang berisi pesan kesalahan yaitu “Proporsi ($\hat{p}$) harus berada diantara 0 dan 1.”. Hal ini menunjukkan bahwa validasi pada variabel $\hat{p}$ berjalan dengan baik untuk menangani input yang tidak logis dari pengguna.

BAB VII

PENUTUP

7.1 Kesimpulan

Berdasarkan hasil pengerjaan proyek ini, dapat disimpulkan bahwa kelompok kami telah berhasil membuat dan menjalankan seluruh program yang direncanakan dengan baik. Enam program yang menjadi studi kasus sukses diterapkan ke dalam bahasa pemrograman R melalui RStudio dan bahasa Python melalui Google Colaboratory. Semua program tersebut mampu menyelesaikan berbagai tugas komputasi dan analisis data secara tepat, mulai dari penghitungan jumlah kata dan kalimat, pengolahan format teks, pencarian akar kuadrat dengan rumus ABC, pemisahan nomor identitas NIP ASN, penentuan kelompok cluster tiga dimensi, hingga perhitungan statistik untuk interval konfidensi.

Selain itu, hasil implementasi menunjukkan adanya kesamaan logika berpikir yang kuat antara bahasa R dan Python meskipun keduanya memiliki aturan penulisan kode atau sintaksis yang berbeda. Melalui penggunaan struktur logika yang serupa, seperti percabangan kondisi (*if-else*) dan perulangan (*looping*), kedua bahasa pemrograman ini terbukti mampu mengeksekusi urutan perintah dengan alur yang sama. Kesamaan logika ini dibuktikan dari hasil keluaran atau *output* data dari kedua lingkungan kerja tersebut yang selalu konsisten, serasi, dan akurat untuk setiap skenario pengujian yang diberikan.

Terakhir, penerapan fungsi validasi atau pembatasan input di dalam struktur kode terbukti sangat efektif untuk menjaga keandalan program agar tidak mudah mengalami kesalahan (*error*). Program mampu secara cerdas mendeteksi dan menyaring masukan data yang tidak logis dari pengguna, seperti mengidentifikasi nilai diskriminan negatif pada rumus kuadrat, menolak ketidaksesuaian batas tanggal lahir pada nomor NIP, serta mendeteksi nilai proporsi sampel yang berada di luar rentang standar. Dengan adanya pembatasan ini, program dapat berjalan dengan lebih aman dan mampu memberikan pesan peringatan yang jelas kepada pengguna apabila terjadi kesalahan pengisian data.

7.2 Kendala

Selama proses pengerjaan proyek ini, terdapat beberapa kendala teknis yang dihadapi oleh kelompok kami, yaitu:

1. Format karakter tanda baca. Pada Program 2, pengaturan tanda petik tunggal maupun ganda di dalam *string* cukup membingungkan. Kelompok kami sempat mengalami

syntax error karena perbedaan cara bahasa R dan Python dalam membaca karakter khusus atau tanda garis miring terbalik (`\`).

2. Perbedaan perhitungan koordinat. Pada Program 5, bahasa R dapat langsung menghitung pengurangan kuadrat koordinat secara sekaligus. Sementara pada Python, operasi tersebut tidak bisa langsung dilakukan, sehingga kelompok harus menuliskan indeks koordinat secara manual satu per satu (`[0]`, `[1]`, `[2]`).
3. Logika validasi kalender secara manual. Pada Program 4, penentuan batas maksimal hari dalam sebulan (terutama untuk bulan Februari pada tahun kabisat) memerlukan penyusunan kondisi bersyarat yang tepat. Hal ini menjadi tantangan karena aturan validasi harus didefinisikan satu per satu secara manual tanpa bantuan pustaka penanggalan (*date library*) bawaan dari sistem.
4. Keterbatasan pilihan parameter. Beberapa input program masih bersifat kaku. Sebagai contoh, pada Program 6, pilihan nilai alpha hanya dibatasi pada angka 5% dan 10% karena nilai tabel Z dimasukkan secara manual ke dalam kode.

7.3 Saran

Untuk pengembangan dan penyempurnaan program ke depannya, beberapa saran yang dapat dilakukan adalah:

1. Penambahan fitur pencegah *crash* (*error handling*). Disarankan untuk menambahkan perintah seperti try-except pada Python atau penanganan sejenis pada R. Fitur ini berguna agar program tidak langsung menutup secara paksa jika pengguna salah memasukkan tipe data (misalnya memasukkan huruf pada kolom angka).
2. Penggunaan sistem pemotong kata yang lebih fleksibel. Pada Program 1, sistem hitung kata dapat ditingkatkan agar pemotongan teks tidak hanya terpaku pada karakter spasi saja, melainkan juga bisa mendeteksi kata yang dipisahkan oleh tanda hubung atau tanda baca lainnya.
3. Otomatisasi nilai tabel statistik. Pada Program 6, keterbatasan nilai alpha dapat diatasi dengan memanfaatkan fungsi tabel Z bawaan dari masing-masing bahasa (seperti `qnorm` di R). Hal ini akan membuat program dapat menghitung tingkat kepercayaan berapa pun secara otomatis.
4. Penerapan validasi panjang karakter input. Pada Program 2, disarankan untuk dilengkapi fitur pengecekan jumlah karakter. Tujuannya agar program dapat memvalidasi input 18 digit, serta memberikan peringatan dan meminta input ulang apabila jumlah digit yang dimasukkan tidak sesuai dengan batas ketentuan.

LAMPIRAN

1. Link Github:

<https://github.com/lanangharsono/Tugas-Proyek-Alpro-Kelompok-G.git>

2. Link Video Demo:

https://drive.google.com/drive/folders/1Yn2F42c2MKw_R14B8H1RtL9J2E2VK-5P?usp=drive_link

3. Link Penggunaan Ai:

https://gemini.google.com/share/d/1t1SV5ruZ4nob_5pZce_QmxgvPPG94ek2?usp=sharing

https://gemini.google.com/share/d/18coIspJm8KjNvZ0a2BAXAE_jnxYEHnn6?usp=sharing

<https://gemini.google.com/share/6c598d9d7d51>

<https://share.gemini.google/EOxKGSTnqKnA>

https://chatgpt.com/s/t_6a3e6b06fc9081919240f0fc4c7d040f